

TravelGo: A Cloud-Powered Real-Time Travel Booking Platform Using AWS

Project Description:

TravelGo is a full-stack, cloud-based travel booking platform designed to simplify the process of reserving buses, trains, flights, and hotels through a unified interface. Built using Flask as the backend framework, the application is deployed on Amazon EC2 and leverages DynamoDB for efficient storage of user data and bookings. TravelGo allows users to register, log in, search for transportation and accommodation options, and book their travel with ease. Once a booking is confirmed or cancelled, users receive real-time email notifications powered by AWS Simple Notification Service (SNS), keeping them informed throughout their journey.

The platform's user-friendly interface supports dynamic seat selection for buses, hotel filtering based on preferences such as luxury or budget, and provides booking summaries along with centralized cancellation management. By combining cloud scalability, responsive design, and secure session handling, TravelGo delivers a seamless and real-time travel planning experience for users.

Scenario 1: Hassle-Free Multi-Mode Travel Booking Experience

TravelGo offers users a unified platform to search and book buses, trains, flights, and hotels all in one place. For instance, a user planning a trip from Hyderabad to Bangalore can log in, select their preferred mode of transport, choose from available options, and proceed to booking. Flask manages the backend operations such as retrieving travel listings and processing user input in real-time. Hosted on AWS EC2, the platform remains responsive even during high-traffic hours like weekends or holiday seasons, allowing multiple users to browse and book without delay.

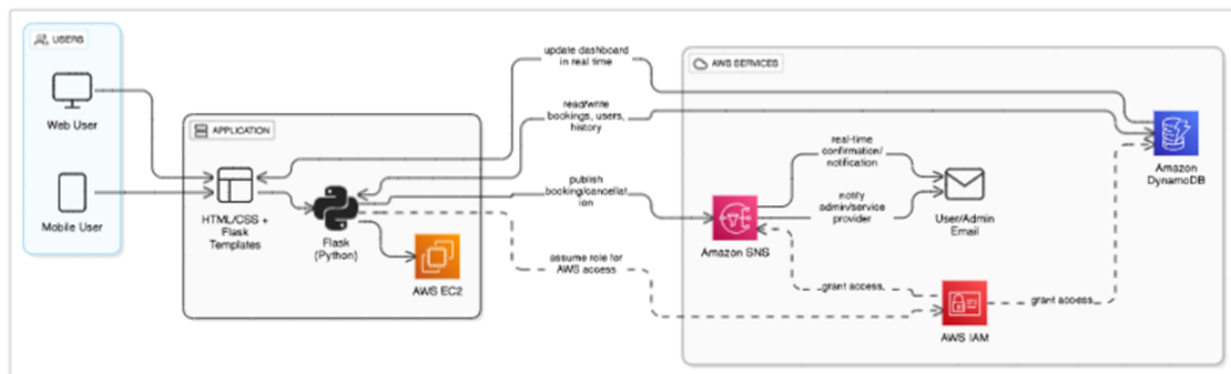
Scenario 2: Real-Time Booking Confirmation with AWS SNS

Once a booking is made—whether it's a train ticket or a hotel stay—TravelGo uses AWS SNS to instantly notify the user. For example, after a student books a hotel in Chennai, SNS sends a real-time email notification confirming the booking with all the relevant details. This notification is triggered from the Flask backend after the booking is successfully recorded in DynamoDB. Additionally, SNS can alert admin or service providers, ensuring transparency and real-time updates on every transaction.

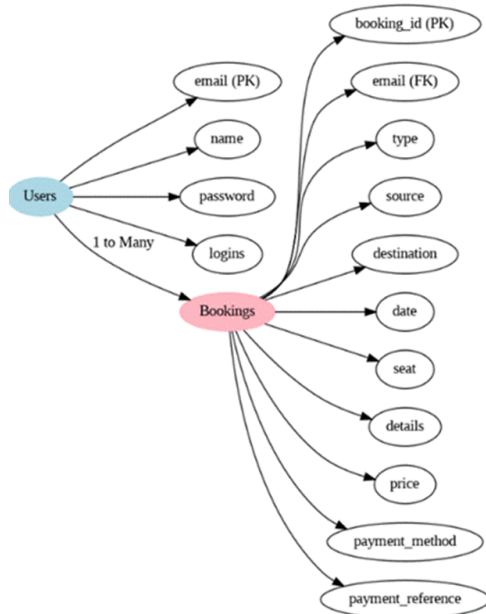
Scenario 3: Dynamic Dashboard with Personal Travel History

TravelGo features a dynamic user dashboard that displays all past and upcoming bookings for the logged-in user. For example, a user who has booked a flight and a hotel can view these bookings categorized by type, along with dates, price, and cancellation options. Flask fetches this data from AWS DynamoDB, which persistently stores all user bookings. The dashboard UI, powered by responsive HTML/CSS and Flask templates, ensures users can review or manage bookings anytime, from any device, with real-time updates and quick cancellation workflows supported.

AWS ARCHITECTURE



Entity Relationship (ER) Diagram:



Pre-requisites:

1. **.AWS Account Setup:**
[AWSAccount Setup](#)
2. **Understanding IAM:**
[IAMOverview](#)
3. **Amazon EC2 Basics:**
[EC2Tutorial](#)
4. **DynamoDB Basics:**
[DynamoDB Introduction](#)
5. **SNS Overview:**
[SNSDocumentation](#)
6. **Git Version Control:**
[Git Documentation](#)

Project WorkFlow:

1. **AWS Account Setup and Login**

Activity 1.1: Set up an AWS account if not already done.

Activity 1.2: Login to AWS Management console.

2. DynamoDB Database Creation and Setup

Activity 2.1: Create a DynamoDB Table.

Activity 2.2: Configure Attributes for User Data and Book Requests.

3. SNS Notification Setup

Activity 3.1: Create SNS topics for book request notifications.

Activity 3.2: Subscribe users and library staff to SNS email notifications.

4. Backend Development and Application Setup

Activity 4.1: Develop the Backend Using Flask.

Activity 4.2: Integrate AWS Services Using boto3.

5. IAM Role Setup

Activity 5.1: Create IAM Role

Activity 5.2: Attach Policies

6. EC2 Instance Setup

Activity 6.1: Launch an EC2 instance to host the Flask application.

Activity 6.2: Configure security groups for HTTP, and SSH access.

7. Deployment on EC2

Activity 6.1: Launch an EC2 instance to host the Flask application.

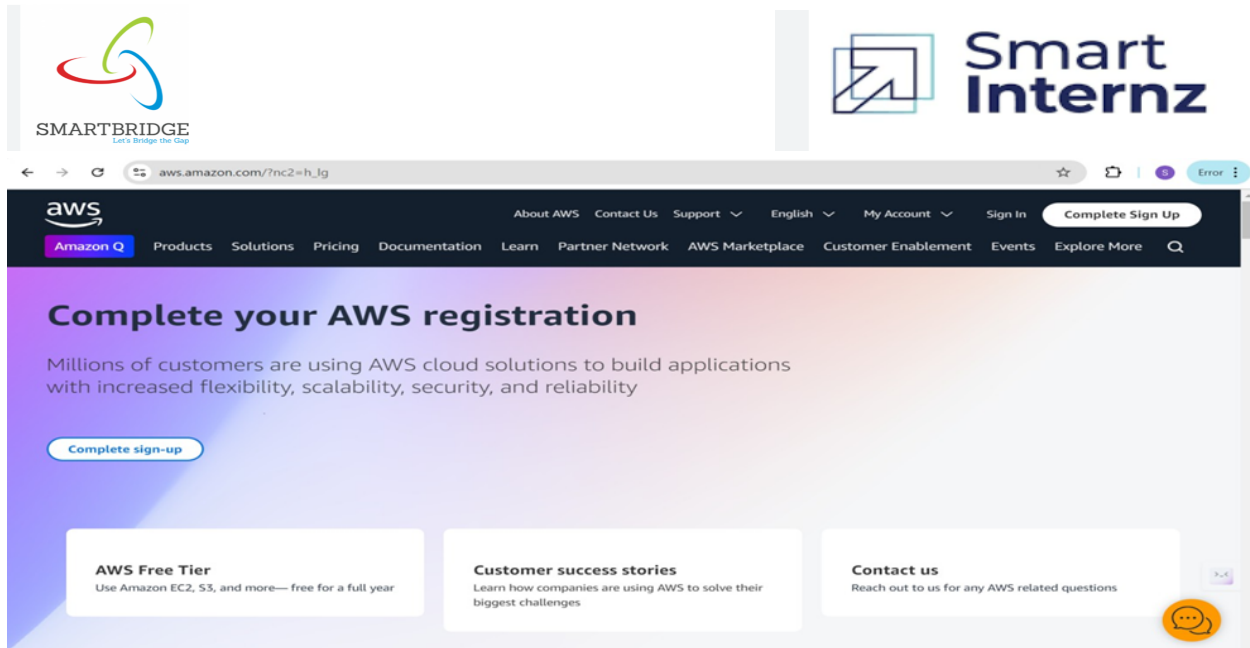
Activity 6.2: configure security groups for Http, and ssh access.

8. Testing and Deployment

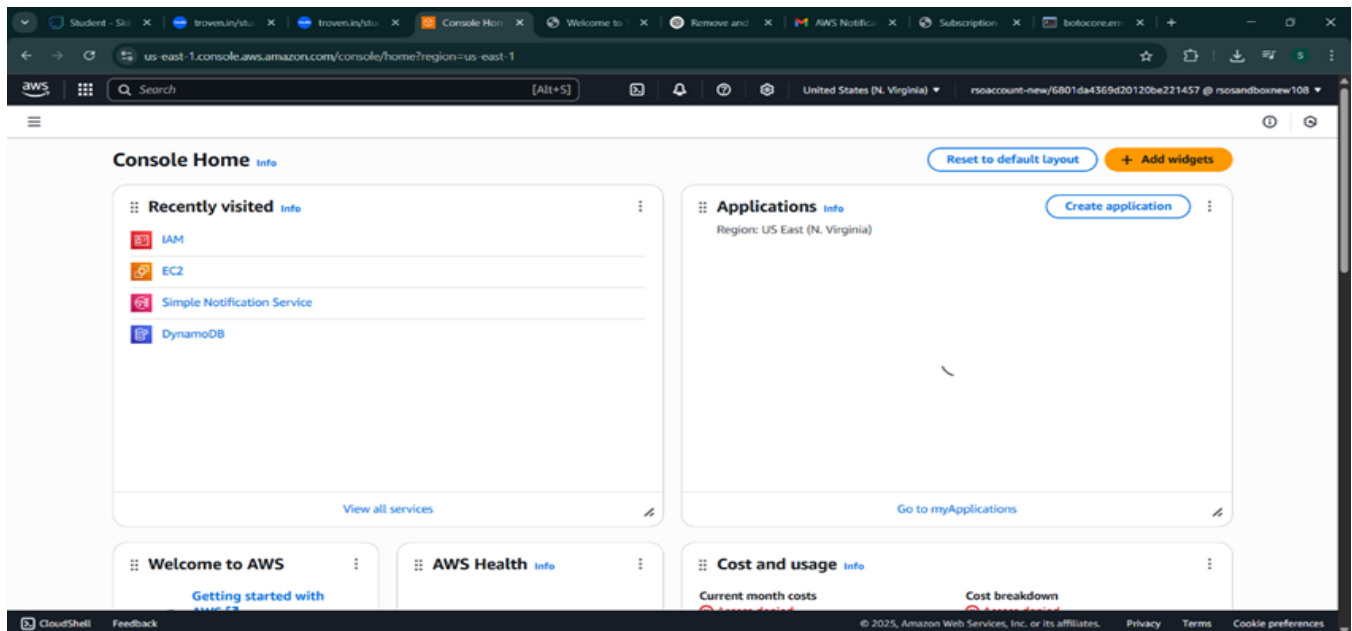
Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.

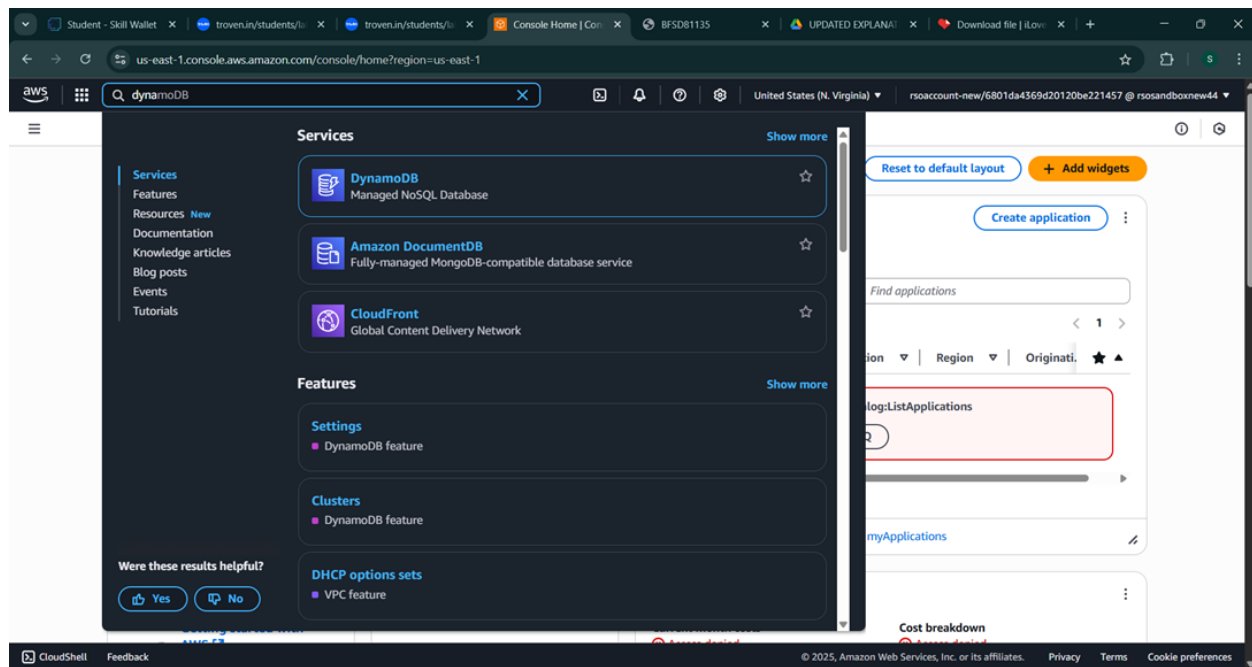
Milestone 1: AWS Account Setup and Login

- **Activity 1.1: Set up an AWS account if not already done**
--> Sign up for an AWS account and configure billing settings.



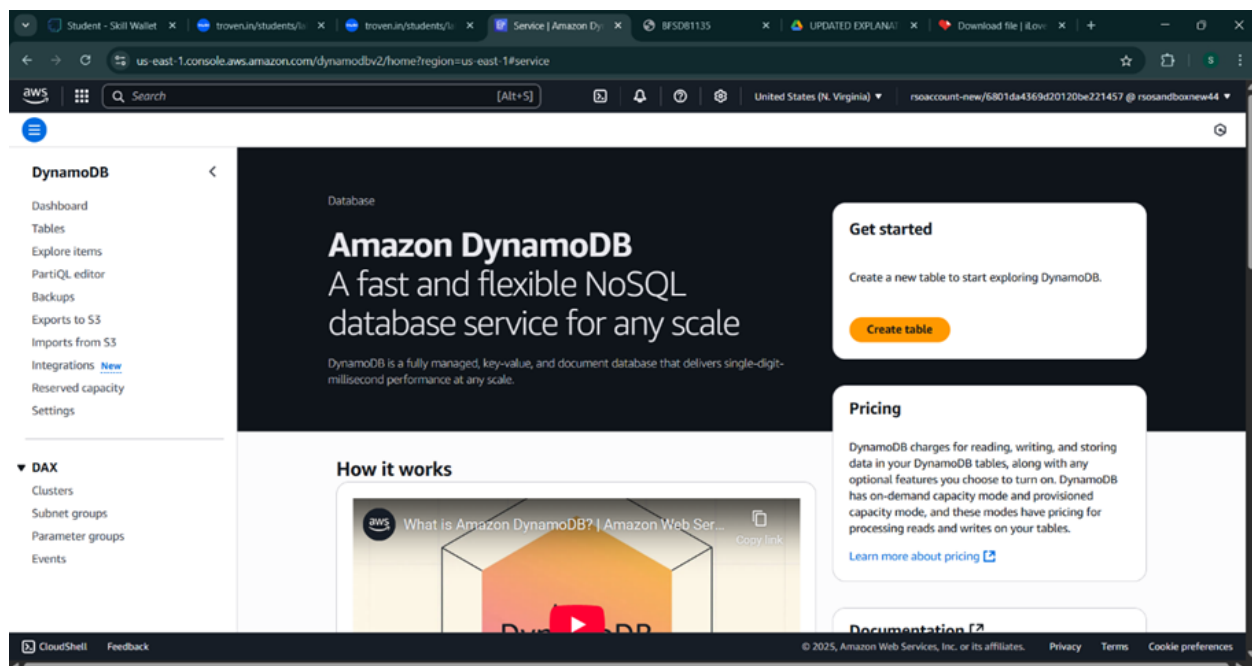
- **Activity 1.2: Log in to the AWS Management Console**
--> After setting up your account, log in to the [AWS Management Console](#).





Milestone 2: DynamoDB Database Creation and Setup

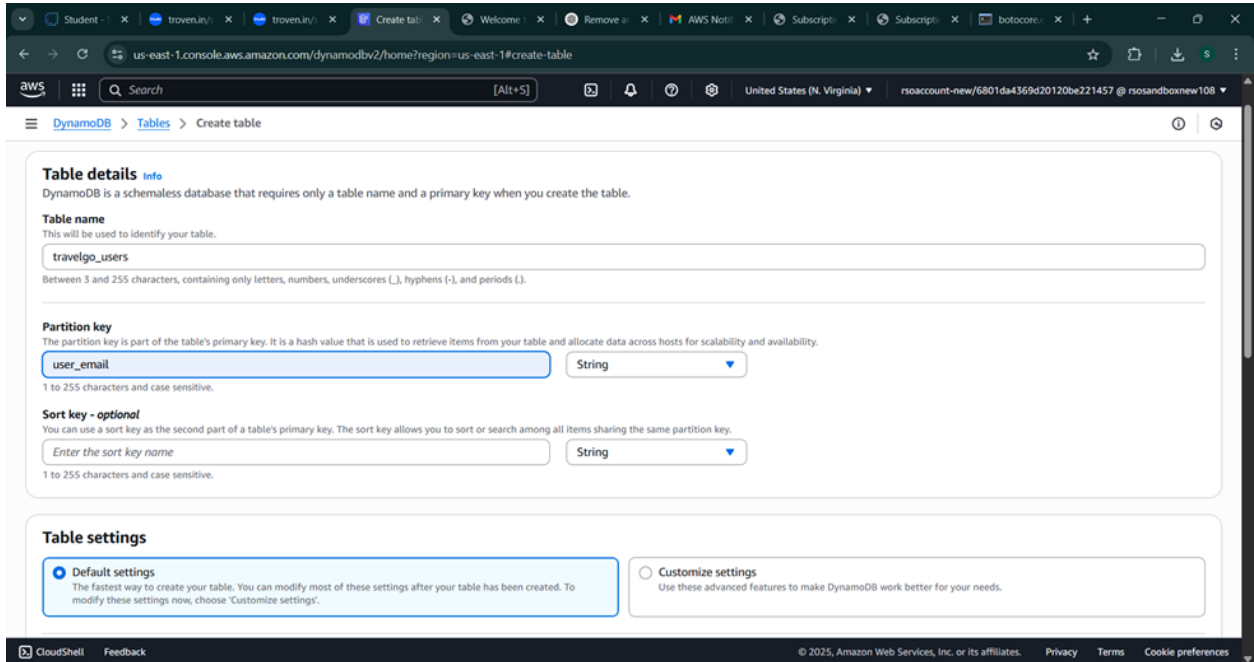
- **Activity 2.1: Navigate to the DynamoDB**
 - In the AWS Console, navigate to DynamoDB and click on create tables.



- **Activity 2.2: Create a DynamoDB table for**

storing registration details and book requests.

- Create Users table with partition key “user_email” with type String and click on create tables.



us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#create-table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

travelgo_users

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

user_email String

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name String

1 to 255 characters and case sensitive.

Table settings

☒ **Default settings**

The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose "Customize settings".

☐ **Customize settings**

Use these advanced features to make DynamoDB work better for your needs.

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



Table class	DynamoDB Standard	Yes
Capacity mode	Provisioned	Yes
Provisioned read capacity	5 RCU	Yes
Provisioned write capacity	5 WCU	Yes
Auto scaling	On	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	Owned by Amazon DynamoDB	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes

Tags

Tags are pairs of keys and optional values, that you can assign to AWS resources. You can use tags to control access to your resources or track your AWS spending.

No tags are associated with the resource.

Add new tag

You can add 50 more tags.

Cancel

Create table

- Follow the same steps to create a requests table with user_email as the primary key for book requests data.

us-east-1.console.aws.amazon.com/dynamodbv2/home#create-table

DynamoDB > Tables > Create table

Table details Info

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

trains

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

email String

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name String

1 to 255 characters and case sensitive.

Table settings

☒ **Default settings**
The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose "Customize settings".

☐ **Customize settings**
Use these advanced features to make DynamoDB work better for your needs.

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/dynamodbv2/home#create-table

DynamoDB > Tables > Create table

Create table

Table details Info

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

bookings

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

booking_date String

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name String

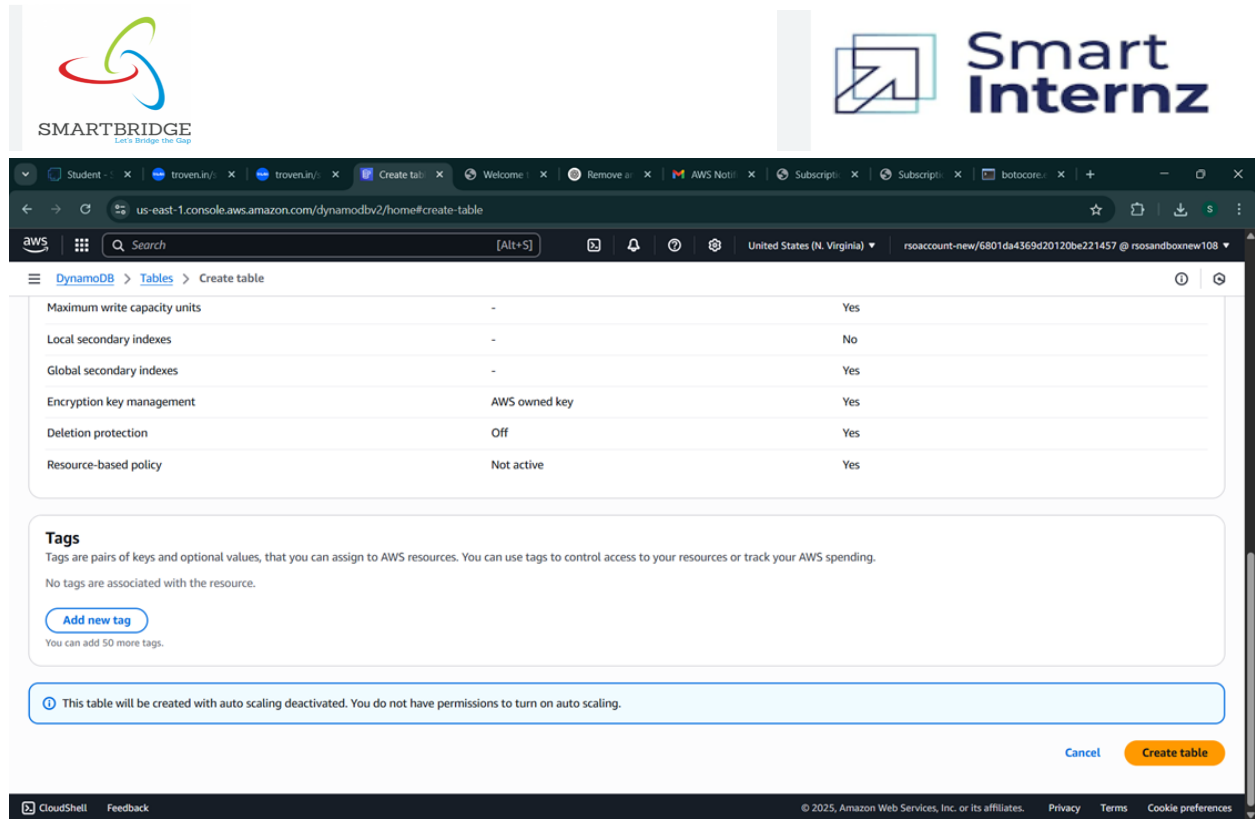
1 to 255 characters and case sensitive.

Table settings

☒ **Default settings**
The fastest way to create your table. You can modify most of these settings after your table has been created. To modify these settings now, choose "Customize settings".

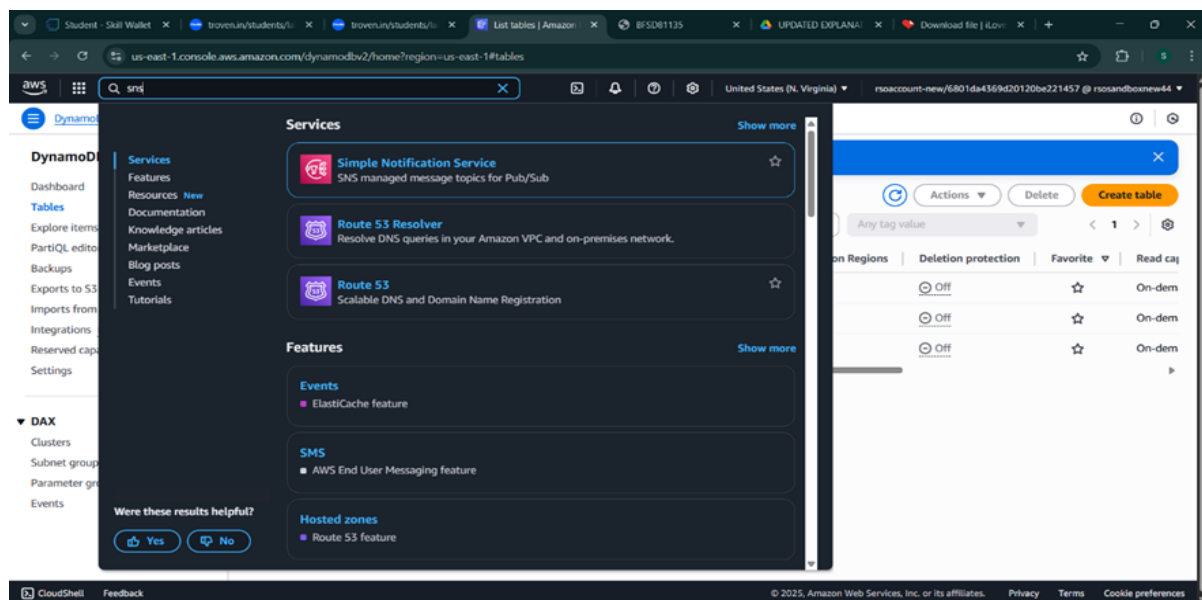
☐ **Customize settings**
Use these advanced features to make DynamoDB work better for your needs.

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



Milestone 3: SNS Notification Setup

● Activity 3.1: Create SNS topics for sending email notifications to users



- In the AWS Console, search for SNS and navigate to the SNS Dashboard.

SMARTBRIDGE

Smart Internz

us-east-1.console.aws.amazon.com/sns/v3/home?region=us-east-1#/homepage

New Feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Application Integration

Amazon Simple Notification Service

Pub/sub messaging for microservices and serverless applications.

Amazon SNS is a highly available, durable, secure, fully managed pub/sub messaging service that enables you to decouple microservices, distributed systems, and event-driven serverless applications. Amazon SNS provides topics for high-throughput, push-based, many-to-many messaging.

Create topic

Topic name
A topic is a message channel. When you publish a message to a topic, it fans out the message to all subscribed endpoints.

Next step

[Start with an overview](#)

Pricing

Amazon SNS has no upfront costs. You pay based on the number of messages you publish, the number of messages you deliver, and any additional API calls for managing topics and...

Benefits and features

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- Click on **Create Topic** and choose a name for the topic.

us-east-1.console.aws.amazon.com/sns/v3/home?region=us-east-1#/create-topic

Amazon SNS > **Topics** > **Create topic**

New Feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Create topic

Details

Type [Info](#)
Topic type cannot be modified after topic is created

☐ FIFO (first-in, first-out)

- Strictly-preserved message ordering
- Exactly-once message delivery
- Subscription protocols: SQS

☒ Standard

- Best-effort message ordering
- At-least-once message delivery
- Subscription protocols: SQS, Lambda, Data Firehose, HTTP, SMS, email, mobile application endpoints

Name

Maximum 256 characters. Can include alphanumeric characters, hyphens (-) and underscores (_).

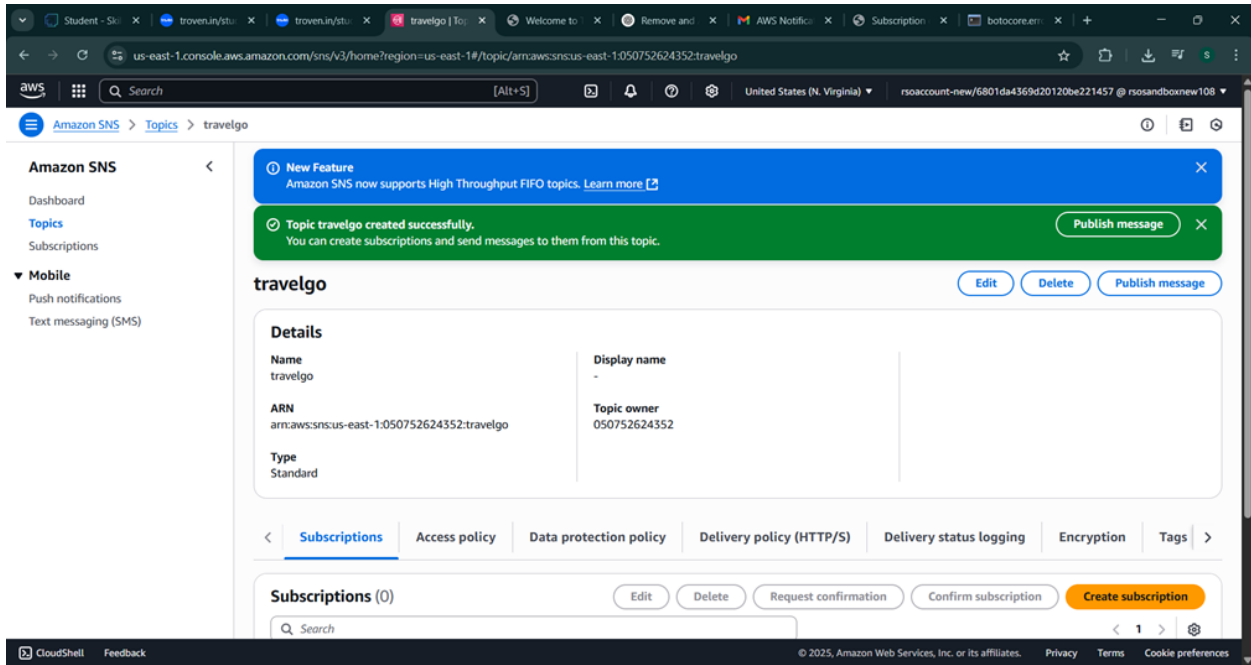
Display name - optional [Info](#)
To use this topic with SMS subscriptions, enter a display name. Only the first 10 characters are displayed in an SMS message.

Maximum 100 characters.

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- Choose Standard type for general notification use cases and Click on Create Topic.



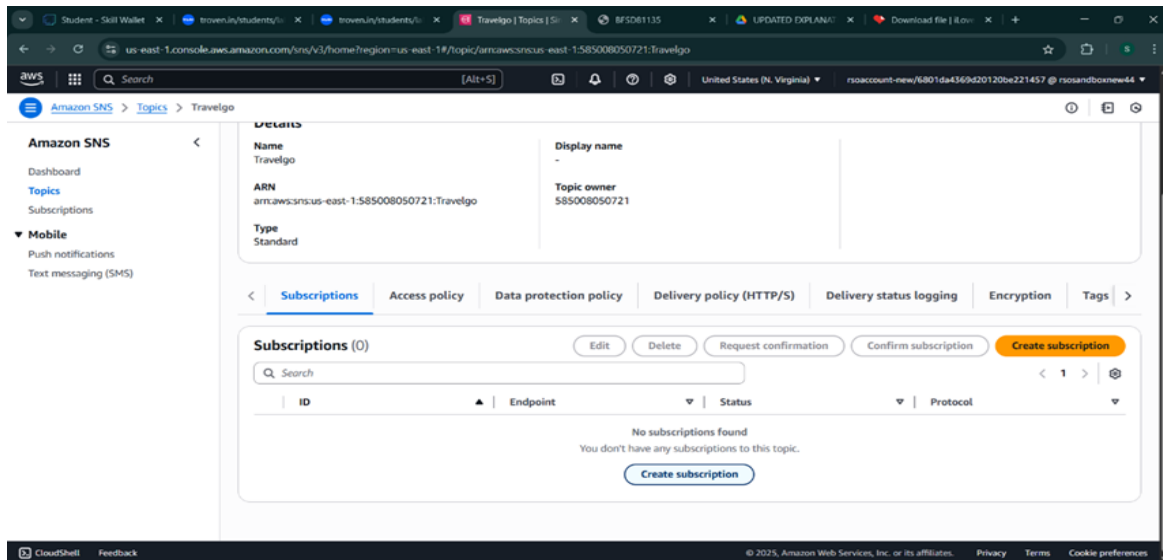
The screenshot shows the Amazon SNS console in the 'travelgo' topic view. A green notification banner at the top states: 'Topic travelgo created successfully. You can create subscriptions and send messages to them from this topic.' The 'Details' section shows the following information:

Name	travelgo	Display name	-
ARN	arn:aws:sns:us-east-1:050752624352:travelgo	Topic owner	050752624352
Type	Standard		

The 'Subscriptions' tab is selected, showing 0 subscriptions. A 'Create subscription' button is visible.

- Configure the SNS topic and note down the **Topic ARN**.

1. Activity 3.2: Subscribe users relevant SNS topics to receive real-time notifications when a book request is made.

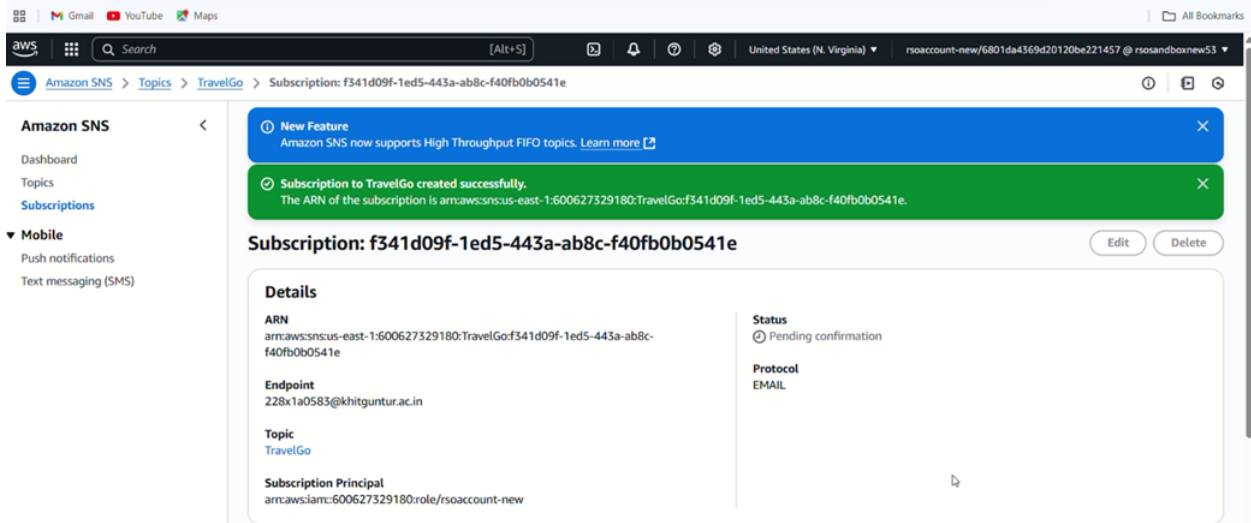


The screenshot shows the Amazon SNS console in the 'travelgo' topic view. The 'Subscriptions' tab is selected, showing 0 subscriptions. The 'Details' section shows the following information:

Name	Travelgo	Display name	-
ARN	arn:aws:sns:us-east-1:585008050721:Travelgo	Topic owner	585008050721
Type	Standard		

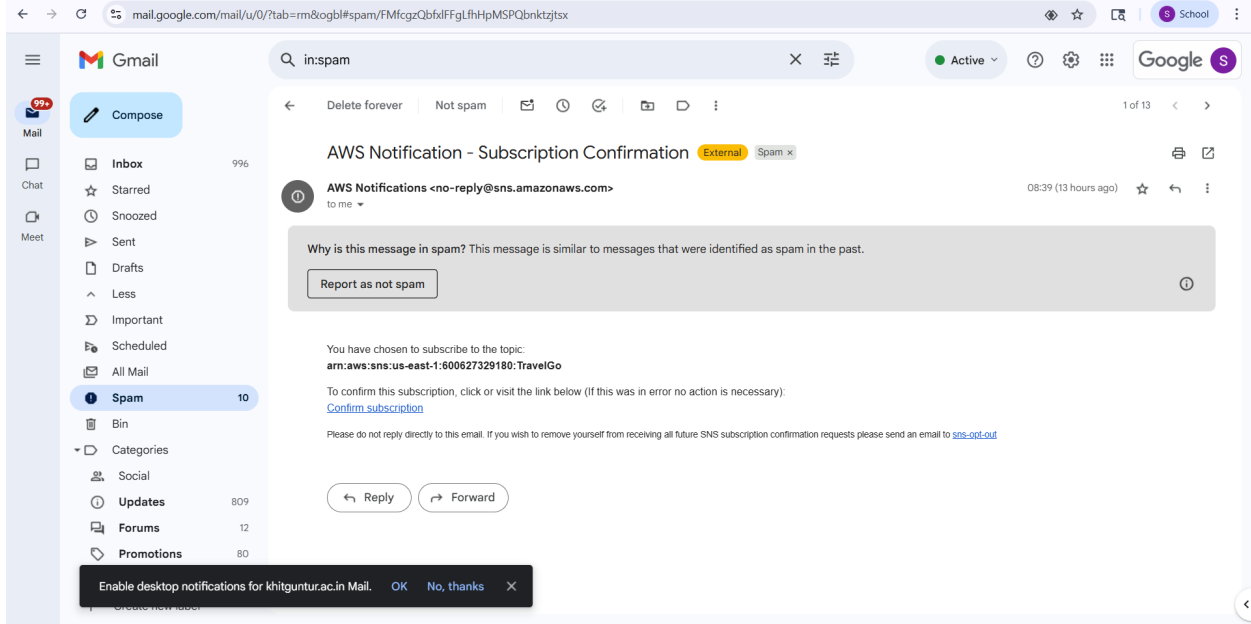
The 'Subscriptions' tab shows a message: 'No subscriptions found. You don't have any subscriptions to this topic.' A 'Create subscription' button is visible.

- Subscribe users to this topic via Email. When a book request is made, notifications will be sent to the subscribed emails.

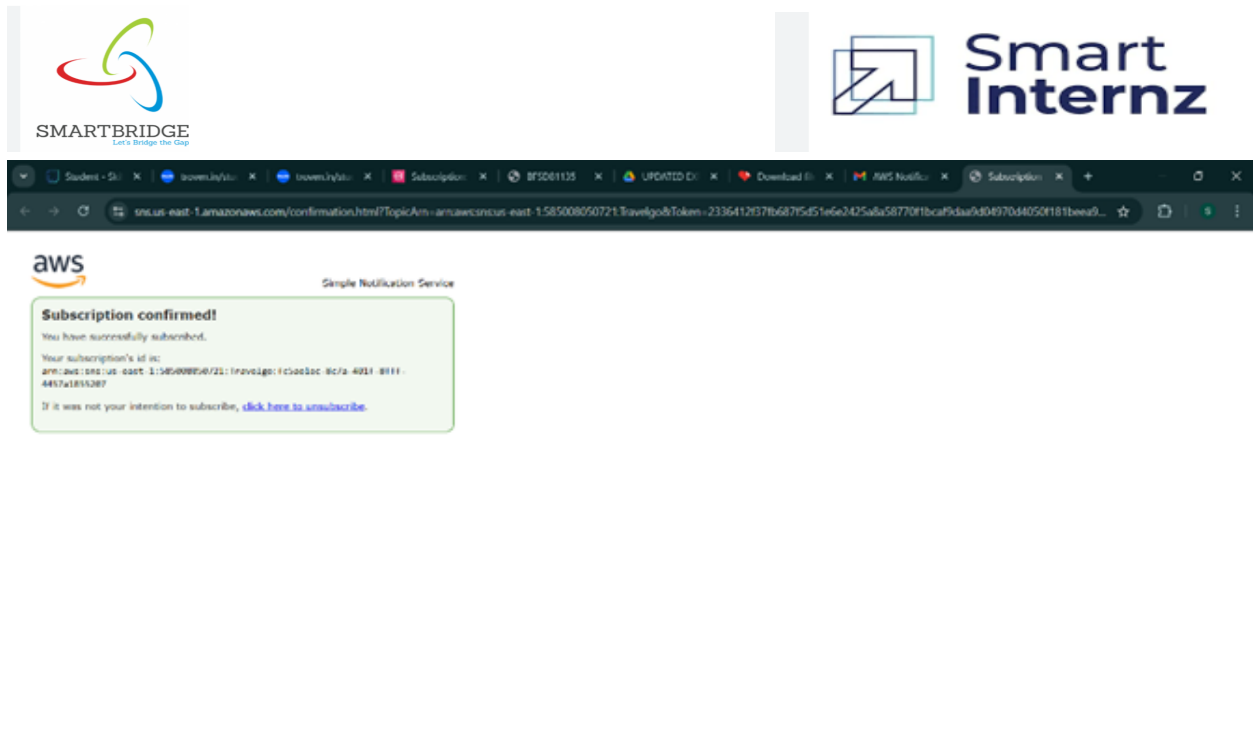


The screenshot shows the Amazon SNS console interface. On the left, the navigation menu includes 'Amazon SNS', 'Dashboard', 'Topics', 'Subscriptions', and 'Mobile'. The main content area displays a 'New Feature' banner about High Throughput FIFO topics, followed by a green success message: 'Subscription to TravelGo created successfully. The ARN of the subscription is arn:aws:sns:us-east-1:600627329180:TravelGo:f341d09f-1ed5-443a-ab8c-f40fb0b0541e.' Below this, the details for the subscription 'f341d09f-1ed5-443a-ab8c-f40fb0b0541e' are shown, including its ARN, endpoint (228x1a0583@khitguntur.ac.in), topic (TravelGo), and status (Pending confirmation).

- After subscription request for the mail confirmation



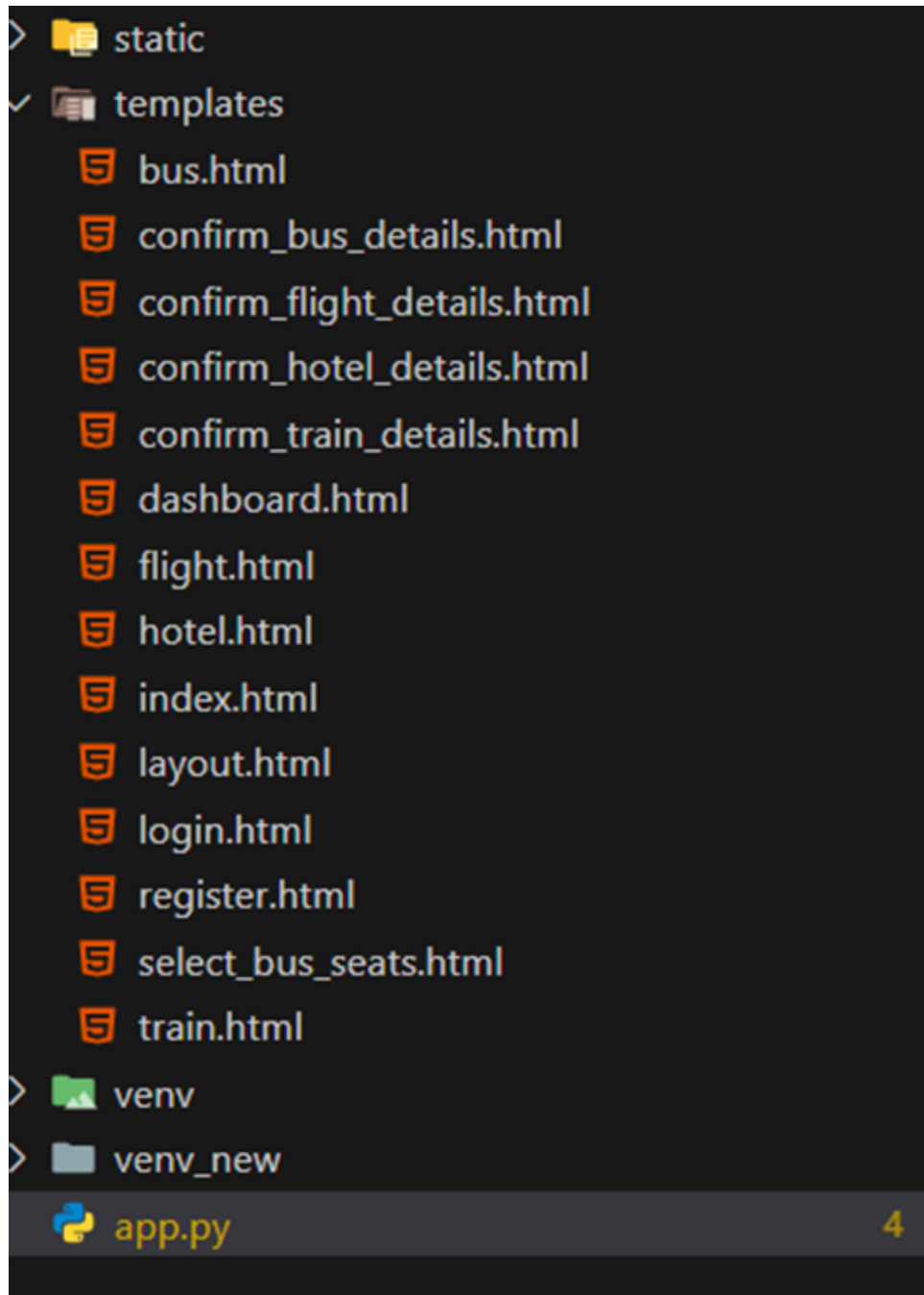
The screenshot shows a Gmail inbox with the search filter 'in:spam'. The email 'AWS Notification - Subscription Confirmation' from 'AWS Notifications <no-reply@sns.amazonaws.com>' is highlighted. It is marked as 'Spam' and 'External'. The email body contains a confirmation link and instructions. A notification banner at the bottom of the email states: 'Why is this message in spam? This message is similar to messages that were identified as spam in the past. Report as not spam'. At the bottom of the screen, a system notification asks to 'Enable desktop notifications for khitguntur.ac.in Mail'.



- Successfully done with the SNS mail subscription and setup, now store the ARN link.

Milestone 4: Backend Development and Application Setup

- **Activity 4.1: Develop the backend using Flask**
 - File Explorer Structure



Description: set up the **TravelGO** project with an app.py file, a static/ folder for assets, and a templates/ directory containing all required HTML pages like home, login, register, booking-specific pages (e.g., bus.html, train.html)

Description of the code :

- **Flask App Initialization**

```
from flask import Flask, render_template, request, redirect, url_for, session, jsonify, flash
import boto3
from boto3.dynamodb.conditions import Key, Attr
from werkzeug.security import generate_password_hash, check_password_hash
from datetime import datetime
from decimal import Decimal
import uuid
import random
```

Description: import essential libraries including Flask utilities for routing, Boto3 for DynamoDB operations,

```
app = Flask(__name__)
```

Description: initialize the Flask application instance using Flask(__name__) to start building the web app.

- **Dynamodb Setup:**

```
2
3 # AWS Setup using IAM Role
4 REGION = 'us-east-1' # Replace with your actual AWS region
5 dynamodb = boto3.resource('dynamodb', region_name=REGION)
6 sns_client = boto3.client('sns', region_name=REGION)
7
8 users_table = dynamodb.Table('travelgo_users')
9 trains_table = dynamodb.Table('trains') # Note: This table is declared bu
0 bookings_table = dynamodb.Table('bookings')
1
```

Description: initialize the DynamoDB resource for the us-east-1 region and set up access to the Users and Requests tables for storing user details and book requests.

(OR)

● MongoDB Setup:

```
app.config['MONGO_DB_URL'] = 'mongodb://Mohan:er2af5Q1UBCewcmZ@cluster0.d16bzx.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0'

# MongoDB connection
client = MongoClient('mongodb+srv://Mohan:er2af5Q1UBCewcmZ@cluster0.d16bzx.mongodb.net/?retryWrites=true&w=majority&appName=Cluster0')
db = client['travel_booking_db']
# Collections
users_collection = db['users']
trains_collection = db['trains']
bookings_collection = db['bookings']
```

● SNS Connection

```
SNS_TOPIC_ARN = 'arn:aws:sns:us-east-1:418272775181:TravelGo:b7d6f5bc-7710-49de-97bc-ddecff5fd023'
topic_arn = SNS_TOPIC_ARN

# Function to send SNS notifications

def send_sns_notification(subject, message):
    try:
        sns_client.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject=subject,
            Message=message
        )
    except Exception as e:
        print(f"SNS Error: Could not send notification - {e}")
        # Optionally, flash an error message to the user or log it more robustly.
```

Description: Configure **SNS** to send notifications when a book request is submitted. Paste your stored ARN link in the `sns_topic_arn` space, along with the `region_name` where the SNS topic is created.

● Routes for Web Pages

● Home Route:

```
# Home route redirects to Registration page
@app.route('/')
def home():
    return redirect(url_for('register'))
```

Description: define the home route `/` to automatically redirect users to the register page when they access the base URL.

- Register Route:

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        # existing = users_collection.find_one({'email': email}) # MongoDB
        existing = users_table.get_item(Key={'email': email}) # DynamoDB
        if 'Item' in existing:
            flash('Email already exists!', 'error')
            return render_template('register.html')
        hashed_password = generate_password_hash(password)
        # users_collection.insert_one({'email': email, 'password': hashed_password}) # MongoDB
        users_table.put_item(Item={'email': email, 'password': hashed_password}) # DynamoDB
        flash('Registration successful! Please log in.', 'success')
        return redirect(url_for('login'))
    return render_template('register.html')
```

Description: define /register route to validate registration form fields, hash the user password using Bcrypt, store the new user in DynamoDB with a login count, and send an SNS notification on successful registration.

- login Route (GET/POST):

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if 'username' in session:
        session.pop('username', None)
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        # user = users_collection.find_one({'email': email}) # MongoDB
        user = users_table.get_item(Key={'email': email}) # DynamoDB
        if 'Item' in user and check_password_hash(user['Item']['password'], password):
            session['email'] = email
            flash('Logged in successfully!', 'success')
            return redirect(url_for('dashboard'))
        else:
            flash('Invalid email or password!', 'error')
            return render_template('login.html')
    return render_template('login.html')
```

Description: define /login route to validate user credentials against DynamoDB, check the password using Bcrypt, update the login count on successful authentication, and redirect users to the home page

- Home, Bus, Train, Flight, Hotel Routes:

```
@app.route('/dashboard')
def dashboard():
    if 'email' not in session:
        return redirect(url_for('login'))

    user_email = session['email']
    # user_bookings = list(bookings_collection.find({'user_email': user_email}).sort('booking_date', -1)) # MongoDB
    response = bookings_table.query(
        KeyConditionExpression=Key('user_email').eq(user_email),
        ScanIndexForward=False
    )
    bookings = response.get('Items', [])
    for booking in bookings:
        if 'total_price' in booking:
            try:
                booking['total_price'] = float(booking['total_price'])
            except Exception:
                booking['total_price'] = 0.0

    for booking in bookings:
        if '_id' in booking and isinstance(booking['_id'], ObjectId):
            booking['_id'] = str(booking['_id'])

        # Determine vehicle_type for display based on booking_type
        if booking.get('booking_type') == 'bus':
            booking['vehicle_type'] = booking.get('type', 'Bus')

        # Add seat information for bus bookings
        if booking.get('selected_seats'):
            booking['seats_display'] = ', '.join(booking['selected_seats'])
        else:
            booking['seats_display'] = 'N/A'

        elif booking.get('booking_type') == 'train':
            booking['vehicle_type'] = f"Train {booking.get('train_number', '')}" if booking.get('train_number') else "Train"
            # Add seat information for train bookings
            if booking.get('selected_seats'):
                booking['seats_display'] = ', '.join(booking['selected_seats'])
            else:
                booking['seats_display'] = 'N/A'

        elif booking.get('booking_type') == 'flight':
            booking['vehicle_type'] = f"Flight {booking.get('flight_number', '')} ({booking.get('airline', '')})"
            # Add seat information for flights
            if booking.get('selected_seats'):
                booking['seats_display'] = ', '.join(booking['selected_seats'])
            else:
                booking['seats_display'] = 'N/A'

        else:
            booking['vehicle_type'] = booking.get('booking_type', 'N/A')

    return render_template('dashboard.html', username=user_email, bookings=bookings)
```

Description: define /dashboard-page to render the main homepage, to handle booking selection and redirection.

1. confirmbooking Routes:

```
@app.route('/train')
def train():
    if 'email' not in session:
        return redirect(url_for('login'))
    return render_template('train.html')

@app.route('/confirm_train_details')
def confirm_train_details():
    if 'email' not in session:
        return redirect(url_for('login'))

    name = request.args.get('name')
    train_number = request.args.get('trainNumber')
    source = request.args.get('source')
    destination = request.args.get('destination')
    departure_time = request.args.get('departureTime')
    arrival_time = request.args.get('arrivalTime')
    price_per_person = float(request.args.get('price'))
    travel_date = request.args.get('date')
    num_persons = int(request.args.get('persons'))
    train_id = request.args.get('trainId')

    total_price = price_per_person * num_persons

    booking_details = {
        'name': name,
        'train_number': train_number,
        'source': source,
        'destination': destination,
        'departure_time': departure_time,
        'arrival_time': arrival_time,
        'price_per_person': Decimal(price_per_person),
        'travel_date': travel_date,
        'num_persons': num_persons,
        'total_price': Decimal(total_price),
        'item_id': train_id,
        'booking_type': 'train',
        'user_email': session['email']
    }
    session['pending_booking'] = booking_details
    return render_template('confirm_train_details.html', booking=booking_details)
```

Description: define /request-form route to capture book request details from users, store the request in DynamoDB, send a thank-you email to the user, notify the admin,

and confirmsubmission with a success message.

Exit Route:

```
@app.route('/logout')
def logout():
    session.pop('email', None)
    flash('You have been logged out.', 'info')
    return redirect(url_for('index'))
```

Description: define /logout route the index.html page to render when the user chooses to leave or close the application.

Deployment Code:

```
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0')
```

Description: start the Flask server to listen on all network interfaces (0.0.0.0) with debug mode enabled for development and testing.

Milestone 5: IAM Role Setup

● Activity 5.1: Create IAM Role.

- In the AWS Console, go to IAM and create a new IAM Role for EC2 to interact with DynamoDB and SNS.

us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles

Search IAM

Dashboard

Access management

- User groups
- Users
- Roles**
- Policies
- Identity providers
- Account settings
- Root access management [New](#)

Access reports

- Access Analyzer
- Resource analysis [New](#)
- Unused access
- Analyzer settings
- Credential report
- Organization activity

CloudShell Feedback

Roles (8) [Info](#)

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

☐	Role name	Trusted entities	Last activity
☐	AWSServiceRoleForAPIGateway	AWS Service: ops.apigateway (Service-	-
☐	AWSServiceRoleForOrganizations	AWS Service: organizations (Service-	216 days ago
☐	AWSServiceRoleForSSO	AWS Service: sso (Service-Linked Rol	-
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Linker	-
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service	-
☐	DCEPrincipal-bunnytf	Account: 058264256896	-
☐	OrganizationAccountAccessRole	Account: 058264256896	15 hours ago
☐	rsoaccount-new	Account: 058264256896	-

[Delete](#) [Create role](#)

Roles Anywhere [Info](#)

Authenticate your non AWS workloads and securely provide access to AWS services.

[Manage](#)

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles/create

Search

IAM > Roles > Create role

Step 1: **Select trusted entity**

Step 2: Add permissions

Step 3: Name, review, and create

Select trusted entity [Info](#)

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

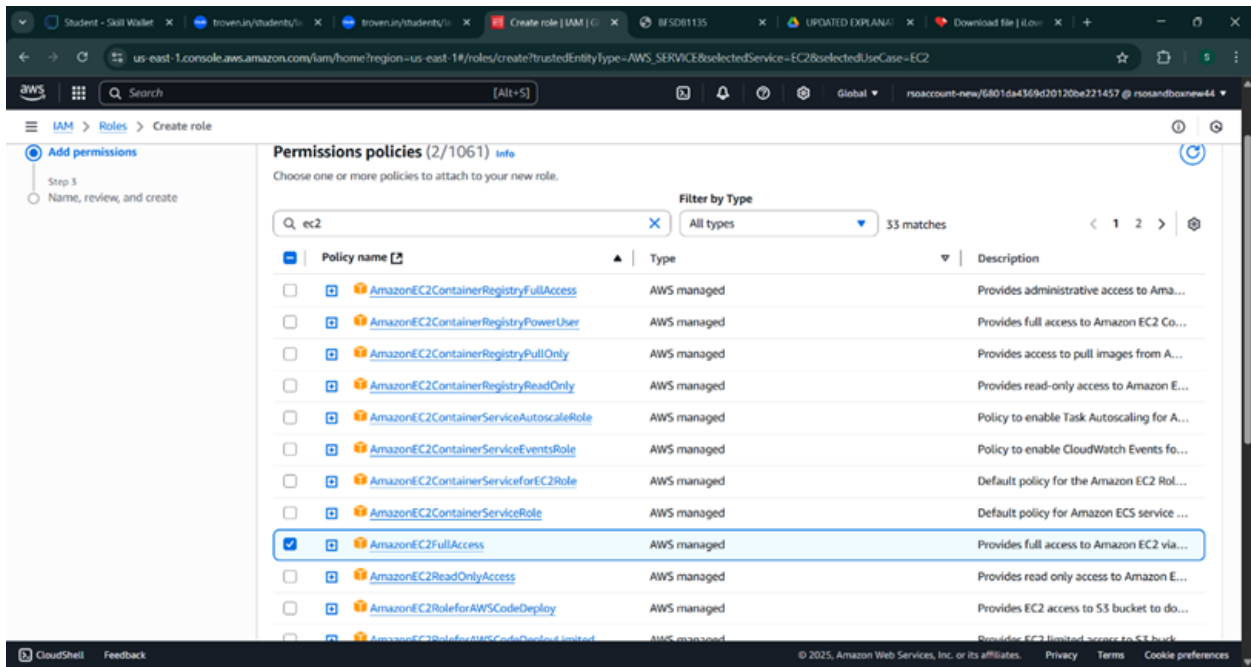
Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
[Choose a service or use case](#)

[Cancel](#) [Next](#)

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)



The screenshot shows the AWS IAM console 'Create role' page. The 'Add permissions' step is active. The 'Permissions policies' section shows a search for 'ec2' with 33 matches. The 'AmazonEC2FullAccess' policy is selected.

Policy name	Type	Description
☐ AmazonEC2ContainerRegistryFullAccess	AWS managed	Provides administrative access to Ama...
☐ AmazonEC2ContainerRegistryPowerUser	AWS managed	Provides full access to Amazon EC2 Co...
☐ AmazonEC2ContainerRegistryPullOnly	AWS managed	Provides access to pull images from A...
☐ AmazonEC2ContainerRegistryReadOnly	AWS managed	Provides read-only access to Amazon E...
☐ AmazonEC2ContainerServiceAutoscaleRole	AWS managed	Policy to enable Task Autoscaling for A...
☐ AmazonEC2ContainerServiceEventsRole	AWS managed	Policy to enable CloudWatch Events fo...
☐ AmazonEC2ContainerServiceforEC2Role	AWS managed	Default policy for the Amazon EC2 Ro...
☐ AmazonEC2ContainerServiceRole	AWS managed	Default policy for Amazon ECS service ...
☒ AmazonEC2FullAccess	AWS managed	Provides full access to Amazon EC2 via...
☐ AmazonEC2ReadOnlyAccess	AWS managed	Provides read only access to Amazon E...
☐ AmazonEC2RoleforAWSCodeDeploy	AWS managed	Provides EC2 access to S3 bucket to do...
☐ AmazonEC2RoleforAWSCodeDeploylimited	AWS managed	Provides EC2 limited access to S3 buck...

● Activity 5.2: Attach Policies.

Attach the following policies to the role:

- AmazonDynamoDBFullAccess: Allows EC2 to perform read/write operations on DynamoDB.
- AmazonSNSFullAccess: Grants EC2 the ability to send notifications via SNS.
- AmazonEC2FullAccess: Allows the user to create a EC2 instance.

us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles/create?trustedEntityType=AWS_SERVICE&selectedService=EC2&selectedUseCase=EC2

Search [Alt+S]

Global rsoaccount-new/6801da4369d20120be221457 @ rsoandboxnew44

IAM > Roles > Create role

Step 1: Select trusted entity
Step 2: **Add permissions**
Step 3: Name, review, and create

Add permissions info

Choose one or more policies to attach to your new role.

Filter by Type: All types 6 matches

Policy name	Type	Description
☒ AmazonDynamoDBFullAccess	AWS managed	Provides full access to Amazon Dynam...
☐ AmazonDynamoDBFullAccess_v2	AWS managed	Provides full access to Amazon Dynam...
☐ AmazonDynamoDBFullAccesswithDataPipeline	AWS managed	This policy is on a deprecation path. Se...
☐ AmazonDynamoDBReadOnlyAccess	AWS managed	Provides read only access to Amazon D...
☐ AWSLambdaDynamoDBExecutionRole	AWS managed	Provides list and read access to Dynam...
☐ AWSLambdaInvocation-DynamoDB	AWS managed	Provides read access to DynamoDB Str...

► Set permissions boundary - optional

Cancel Previous Next

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles/create?trustedEntityType=AWS_SERVICE&selectedService=EC2&selectedUseCase=EC2

Search [Alt+S]

Global rsoaccount-new/6801da4369d20120be221457 @ rsoandboxnew44

IAM > Roles > Create role

Step 1: Select trusted entity
Step 2: **Add permissions**
Step 3: Name, review, and create

Add permissions info

Choose one or more policies to attach to your new role.

Filter by Type: All types 5 matches

Policy name	Type	Description
☒ AmazonSNSFullAccess	AWS managed	Provides full access to Amazon SNS via...
☐ AmazonSNSReadOnlyAccess	AWS managed	Provides read only access to Amazon S...
☐ AmazonSNSRole	AWS managed	Default policy for Amazon SNS service...
☐ AWSElasticBeanstalkRoleSNS	AWS managed	(Elastic Beanstalk operations role) Allo...
☐ AWSIoTDeviceDefenderPublishFindingsToSN...	AWS managed	Provides messages publish access to S...

► Set permissions boundary - optional

Cancel Previous Next

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

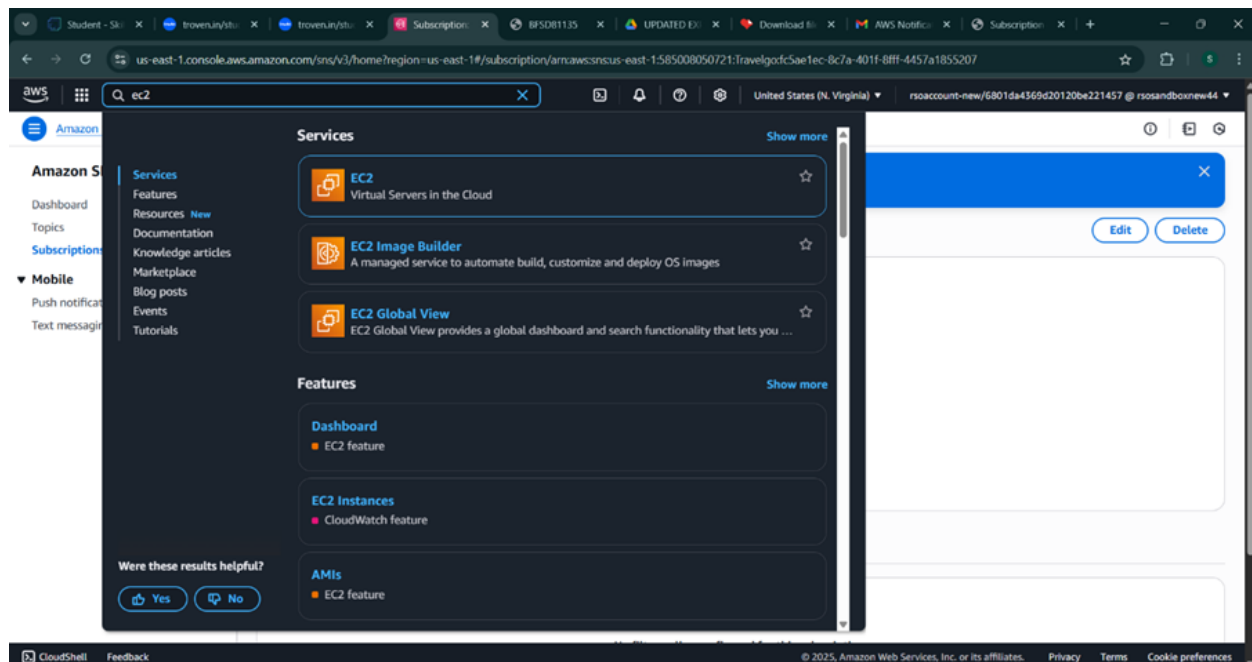
Milestone 6: EC2 InstanceSetup

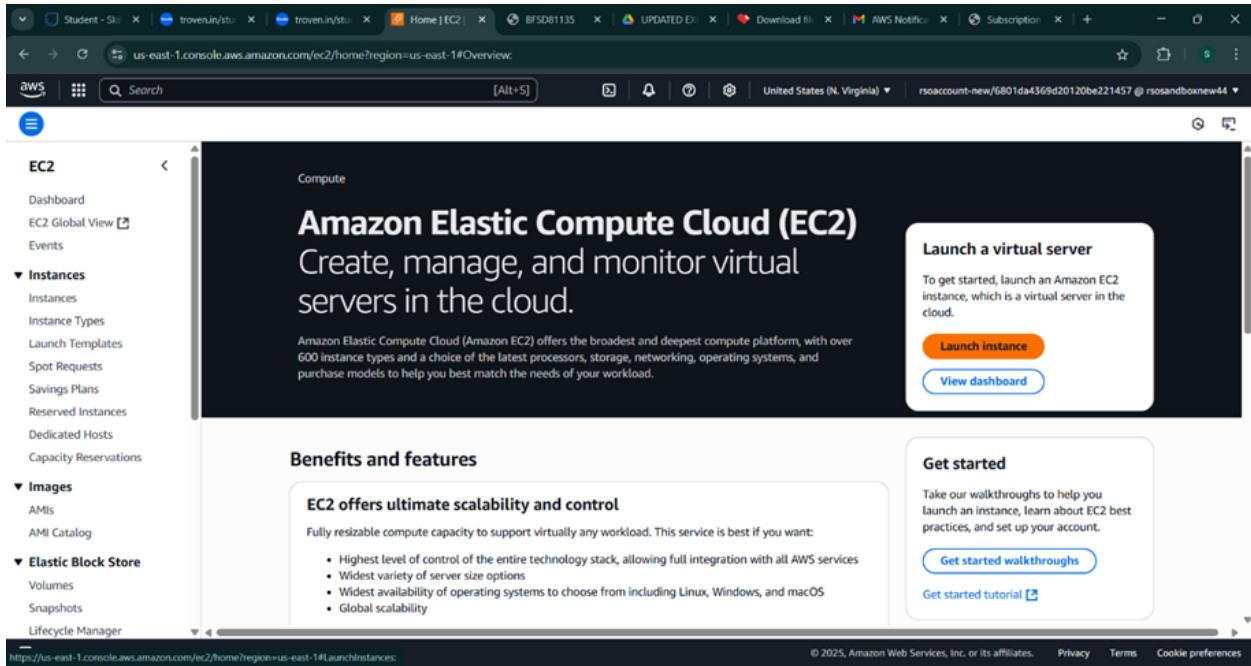
- **Note: Load your Flask app and Html files into GitHub repository.**

static	Added full project files
templates	app.py changed
venv	Added full project files
venv_new	Added full project files
README.md	first commit
app.py	Update app.py

● **Activity 6.1: Launch an EC2 instance to host the Flask application.**

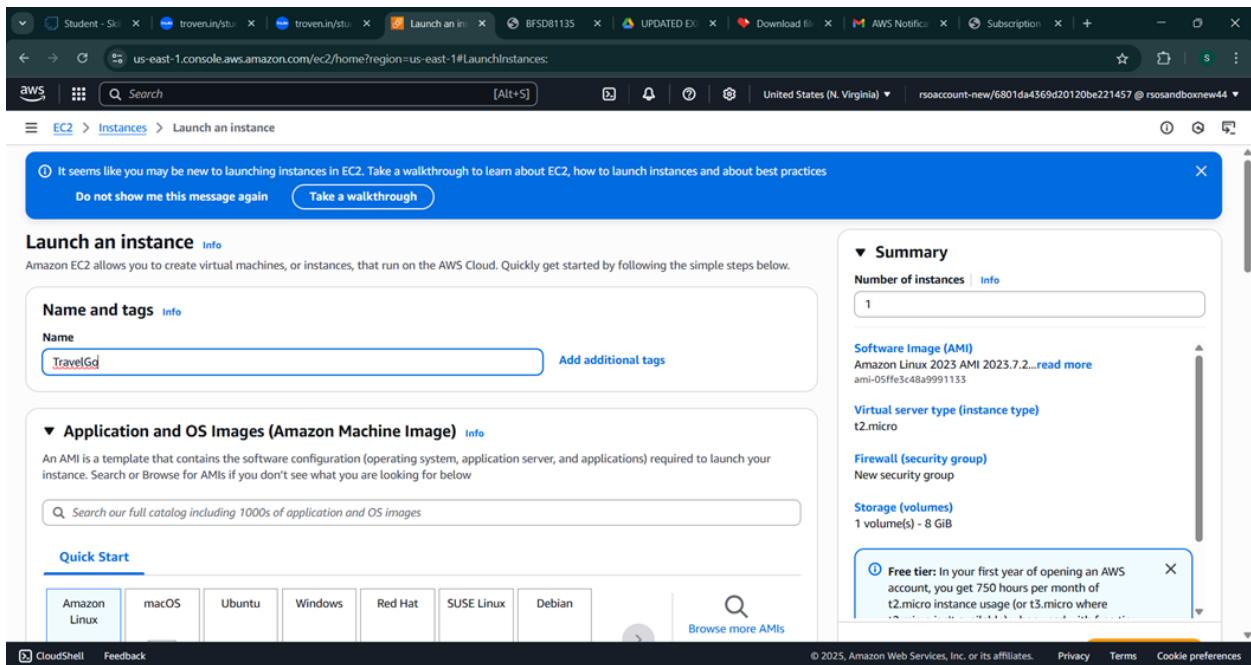
- **Launch EC2 Instance**
 - In the AWS Console, navigate to EC2 and launch a new instance.





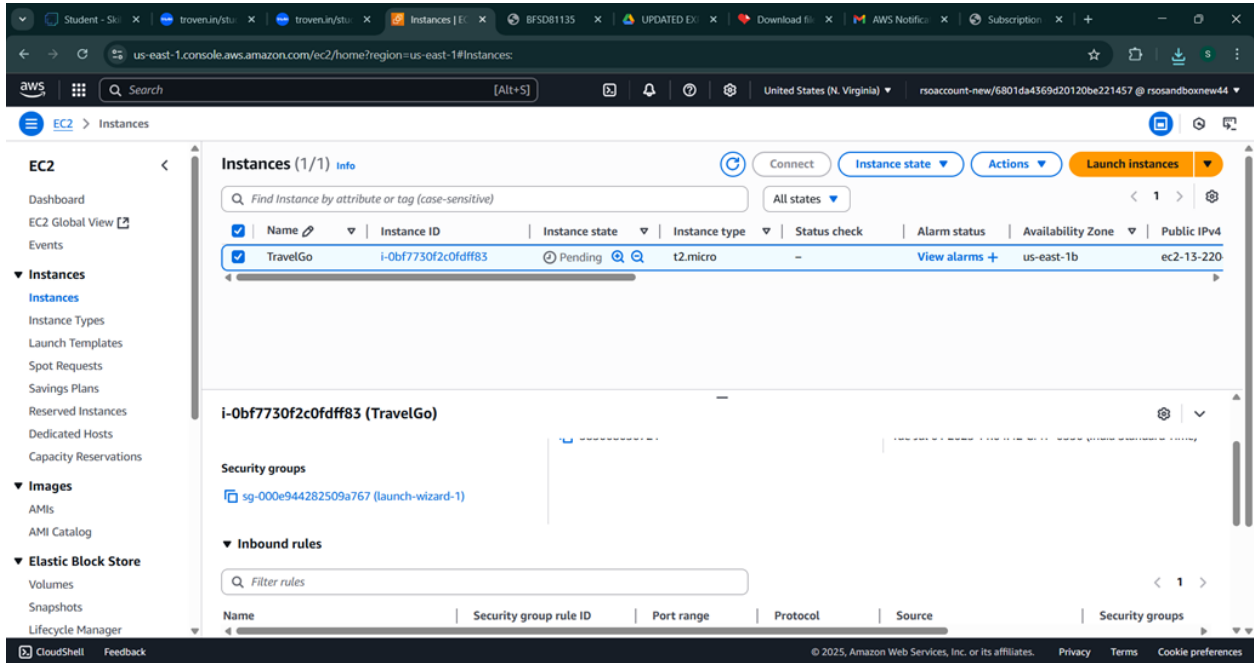
The screenshot shows the AWS Management Console for the EC2 service. The left sidebar contains navigation links for EC2, Dashboard, EC2 Global View, Events, Instances, Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, Capacity Reservations, Images, AMIs, AMI Catalog, Elastic Block Store, Volumes, Snapshots, and Lifecycle Manager. The main content area features a large header with the text "Amazon Elastic Compute Cloud (EC2) Create, manage, and monitor virtual servers in the cloud." Below this, there are sections for "Benefits and features" and "Get started". The "Benefits and features" section highlights that EC2 offers ultimate scalability and control, with a list of benefits including the highest level of control, widest variety of server size options, widest availability of operating systems, and global scalability. The "Get started" section provides links to "Get started walkthroughs" and a "Get started tutorial". A "Launch a virtual server" button is prominently displayed in the top right corner.

- Click on Launchinstance to launchEC2 instance
- Choose Amazon Linux 2 or Ubuntu as the AMI and t2.micro as the instancetype (free-tier eligible).



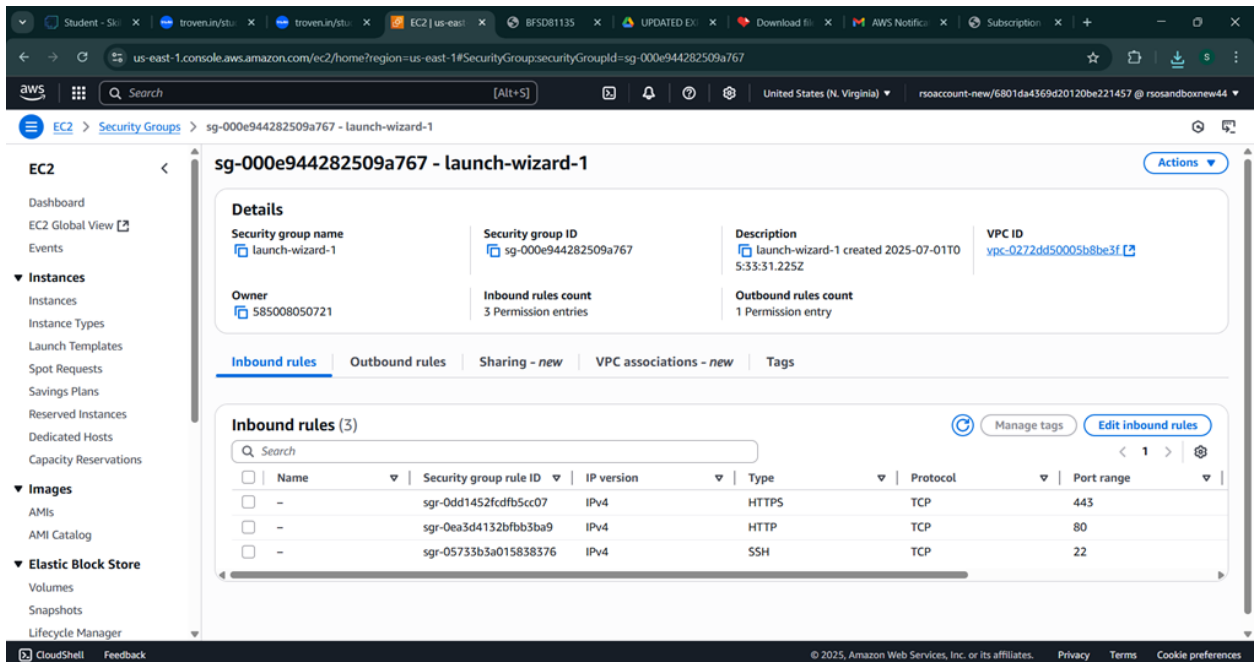
The screenshot shows the "Launch an instance" wizard in the AWS Management Console. The wizard is divided into several sections: "Name and tags", "Application and OS Images (Amazon Machine Image)", "Summary", and "Quick Start". The "Name and tags" section has a text input field for the instance name, which is currently set to "TravelGd". The "Application and OS Images" section provides a search bar for AMIs and a "Quick Start" section with buttons for various operating systems: Amazon Linux, macOS, Ubuntu, Windows, Red Hat, SUSE Linux, and Debian. The "Summary" section on the right provides a overview of the configuration, including the number of instances (1), the software image (AMI), the virtual server type (instance type), the firewall (security group), and the storage (volumes). A "Free tier" notification is also visible at the bottom right, stating that the user gets 750 hours per month of t2.micro instance usage (or t3.micro where applicable) for free in their first year of opening an AWS account.

- Create and download the key pair for Server access.



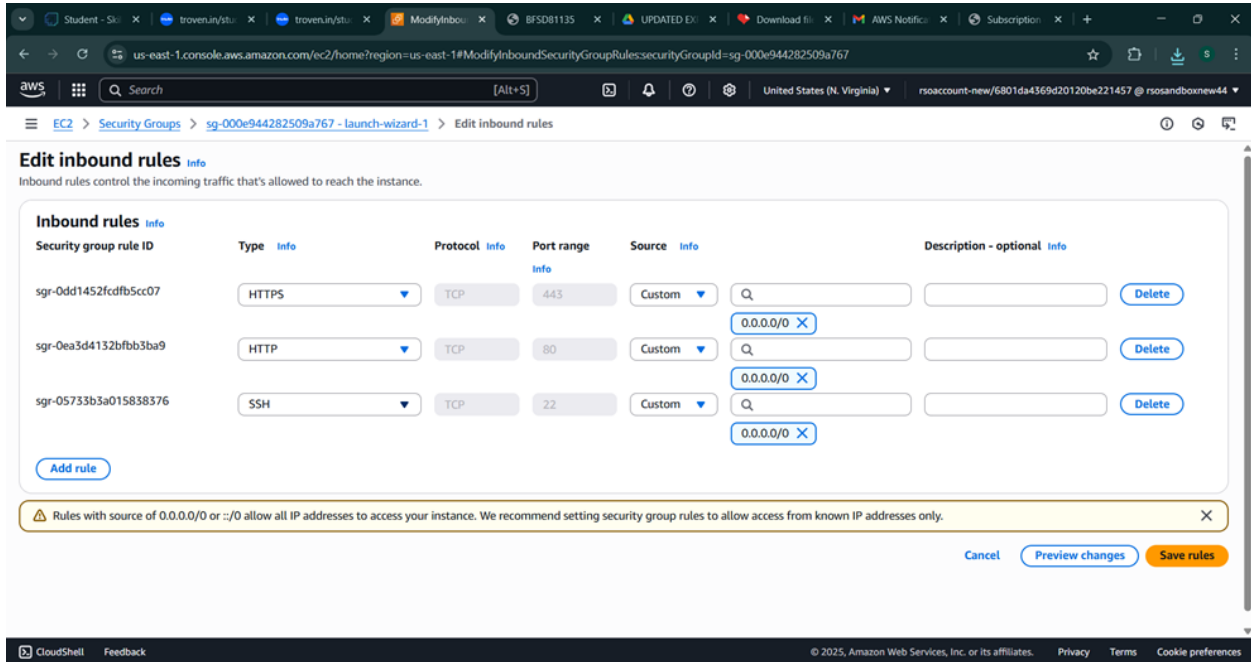
The screenshot shows the AWS Management Console for the 'us-east-1' region. The 'Instances' page is active, displaying a table with one instance named 'TravelGo' (ID: i-0bf7730f2c0fdff83) in a 'Pending' state. The instance is of type 't2.micro' and is located in the 'us-east-1b' availability zone. Below the table, the details for the selected instance are shown, including its security groups (sg-000e944282509a767) and inbound rules.

- Activity 6.2: Configure security groups for HTTP and SSH access.



The screenshot shows the AWS Management Console for the 'us-east-1' region, specifically the 'Security Groups' page. The selected security group is 'sg-000e944282509a767 - launch-wizard-1'. The 'Inbound rules' tab is active, showing three rules: one for HTTPS (port 443), one for HTTP (port 80), and one for SSH (port 22). The rules are configured for IPv4 and TCP protocol.

● Activity 6.3: Add a custom inbound rule with port range 5000 and source Anywhere IPv4



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ModifyInboundSecurityGroupRules:securityGroupId=sg-000e944282509a767

EC2 > Security Groups > sg-000e944282509a767 - launch-wizard-1 > Edit inbound rules

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

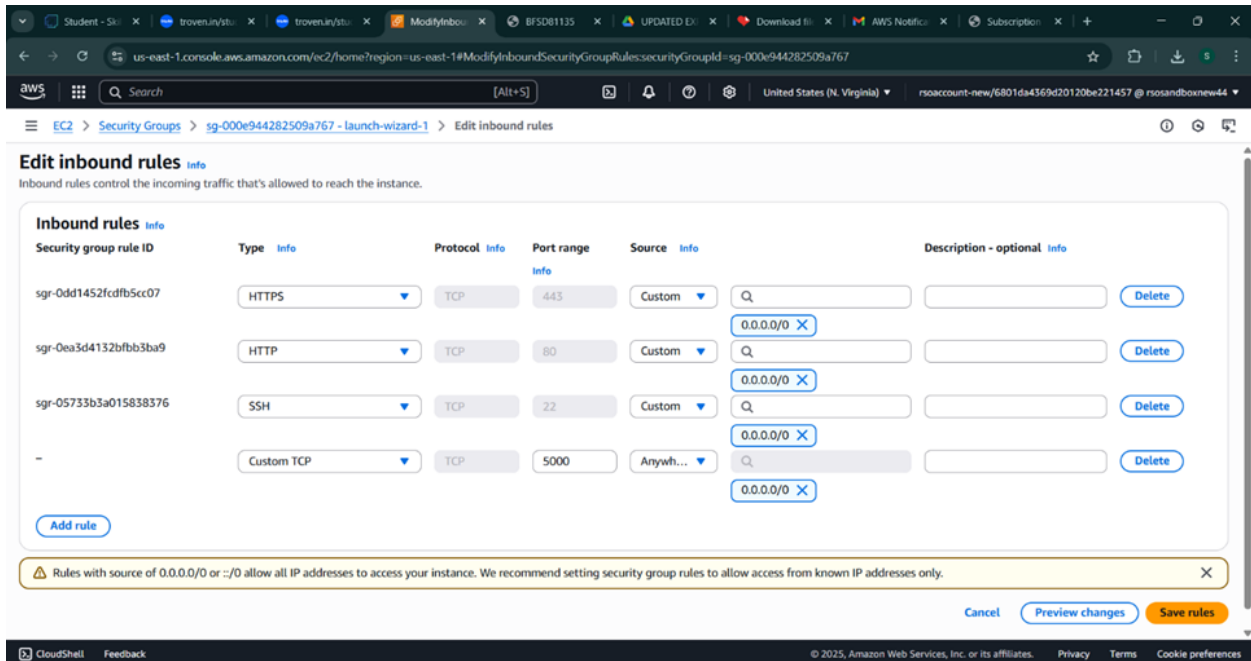
Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sgr-0dd1452cdfb5cc07	HTTPS	TCP	443	Custom	0.0.0.0/0	Delete
sgr-0ea3d4132bfb3ba9	HTTP	TCP	80	Custom	0.0.0.0/0	Delete
sgr-05733b3a015838376	SSH	TCP	22	Custom	0.0.0.0/0	Delete

[Add rule](#)

Rules with source of 0.0.0.0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

[Cancel](#) [Preview changes](#) [Save rules](#)

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ModifyInboundSecurityGroupRules:securityGroupId=sg-000e944282509a767

EC2 > Security Groups > sg-000e944282509a767 - launch-wizard-1 > Edit inbound rules

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sgr-0dd1452cdfb5cc07	HTTPS	TCP	443	Custom	0.0.0.0/0	Delete
sgr-0ea3d4132bfb3ba9	HTTP	TCP	80	Custom	0.0.0.0/0	Delete
sgr-05733b3a015838376	SSH	TCP	22	Custom	0.0.0.0/0	Delete
-	Custom TCP	TCP	5000	Anywhere IPv4	0.0.0.0/0	Delete

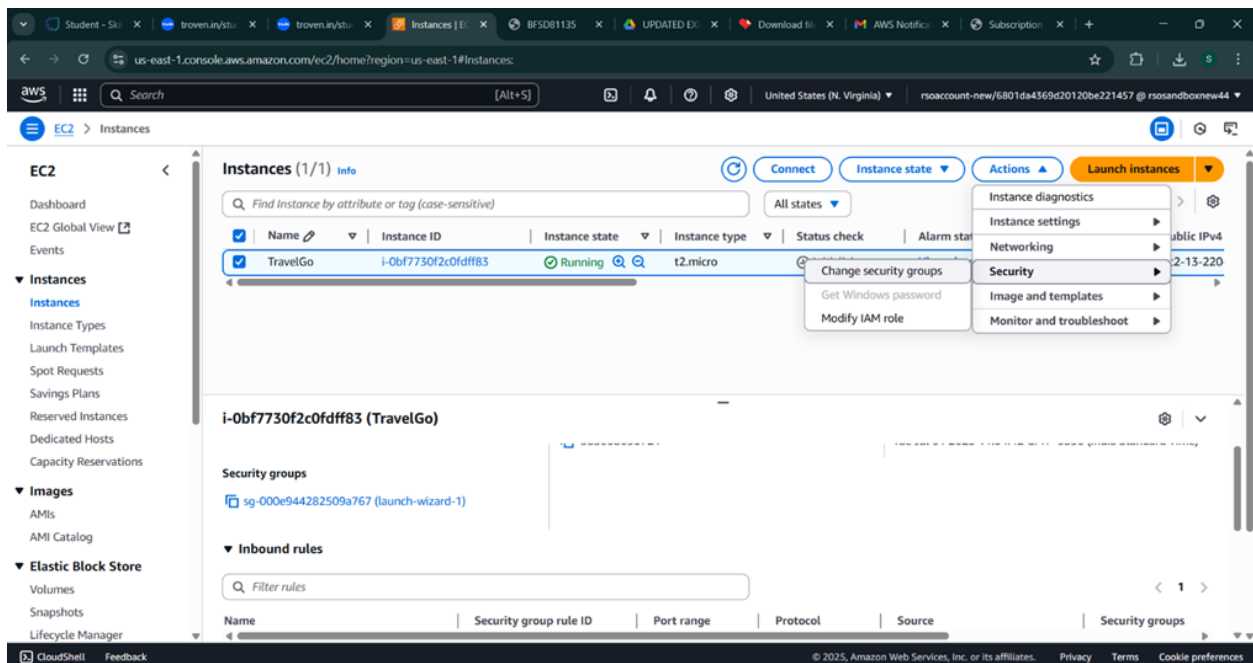
[Add rule](#)

Rules with source of 0.0.0.0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

[Cancel](#) [Preview changes](#) [Save rules](#)

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- To connect to EC2 using **EC2 Instance Connect**, start by ensuring that an **IAM role** is attached to your EC2 instance. You can do this by selecting your instance, clicking on **Actions**, then navigating to **Security** and selecting **Modify IAM Role** to attach the appropriate role. After the IAM role is connected, navigate to the **EC2** section in the **AWS Management Console**. Select the **EC2 instance** you wish to connect to. At the top of the **EC2 Dashboard**, click the **Connect** button. From the connection methods presented, choose **EC2 Instance Connect**. Finally, click **Connect** again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ModifyIAMRoleInstanceID=i-0bf7730f2c0df83

EC2 > Instances > i-0bf7730f2c0df83 > Modify IAM role

Modify IAM role [Info](#)

Attach an IAM role to your instance.

Instance ID
i-0bf7730f2c0df83 (TravelGo)

IAM role
Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.

Choose IAM role

Q

No IAM Role
Choose this option to detach an IAM role

Studentuser
arn:aws:iam::585008050721:instance-profile/Studentuser

Studentuser

ected instance?

Cancel Update IAM role

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- Now connect the EC2 with the files

us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ConnectToInstanceInstanceID=i-0bf7730f2c0df83

EC2 > Instances > i-0bf7730f2c0df83 > Connect to instance

Connect [Info](#)

Connect to an instance using the browser-based client.

EC2 Instance Connect Session Manager **SSH client** EC2 serial console

Instance ID
i-0bf7730f2c0df83 (TravelGo)

1. Open an SSH client.
2. Locate your private key file. The key used to launch this instance is travel-go-2.pem
3. Run this command, if necessary, to ensure your key is not publicly viewable.
chmod 400 "travel-go-2.pem"
4. Connect to your instance using its Public DNS:
ec2-13-220-193-179.compute-1.amazonaws.com

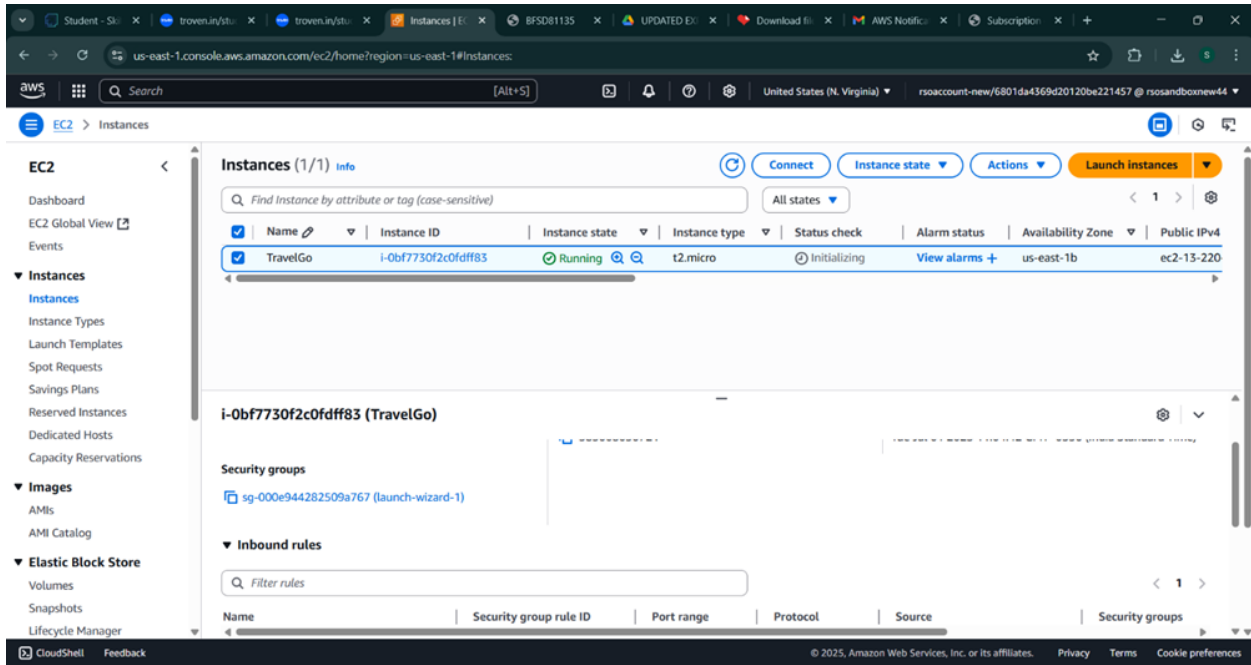
Example:
ssh -i "travel-go-2.pem" ec2-user@ec2-13-220-193-179.compute-1.amazonaws.com

Note: In most cases, the guessed username is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

Cancel

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



The screenshot shows the AWS Management Console for the 'us-east-1' region. The 'Instances' page displays a single instance named 'TravelGo' with ID 'i-0bf7730f2c0fdff83'. The instance is in the 'Running' state and is of type 't2.micro'. The console shows the instance's details, including its security groups and inbound rules.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
TravelGo	i-0bf7730f2c0fdff83	Running	t2.micro	Initializing	View alarms +	us-east-1b	ec2-13-220

Milestone 7: Deployment on EC2

Activity 7.1: Install Software on the EC2 Instance

Install These :

1. python3
2. Flask
3. Git:

On Amazon Linux2:

```
sudo yum update-y
sudo yum install python3 git
sudo yum install flask boto3
```

Verify Installations:

```
flask --version
git --version
```

Activity 7.2: Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

Run: 'git clone <https://github.com/vikram0675/TravelGo.git>'

- This will download your project to the EC2 instance.

To navigate to the project directory, run the following command:

```
cd TravelGo
```

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Run the Flask Application

```
sudo flask run --host=0.0.0.0 --port=80
```

Verify the Flask app is running :

<http://18.205.190.145:5000/>

- Run the Flask app on the EC2 instance

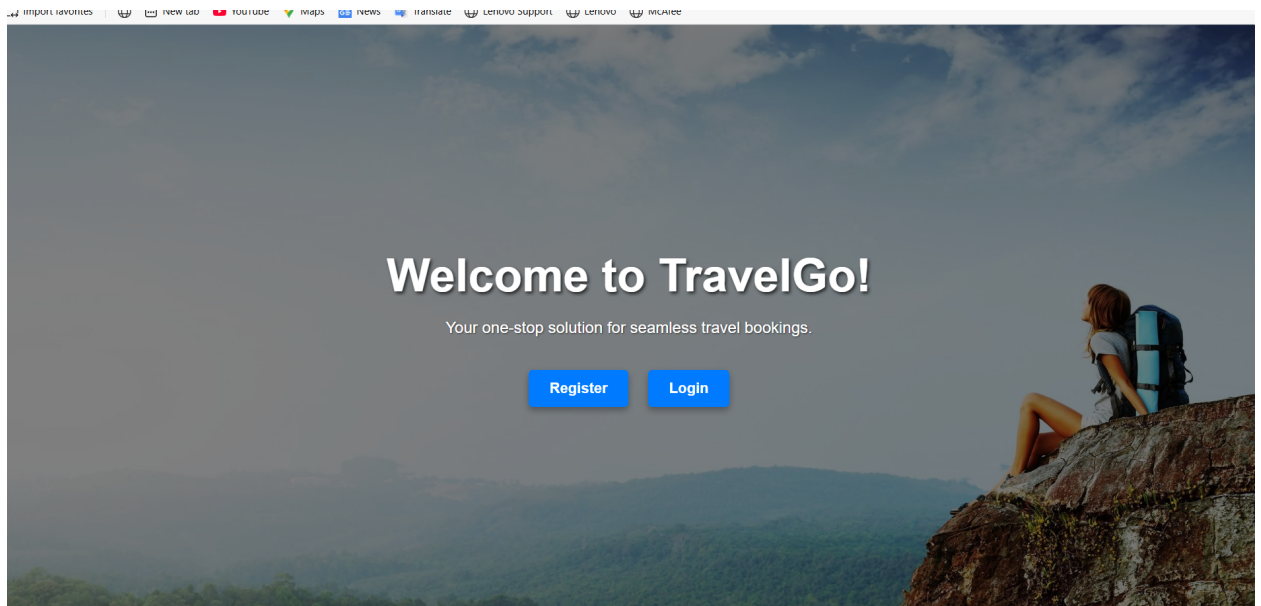
```
ec2-user@ip-172-31-84-183:~$ python3 app.py
.38.46->boto3) (2.8.1)
Requirement already satisfied: urllib3<1.27,>=1.25.4 in /usr/lib/python3.9/site-packages (from botocore<1.39.0,>=1.38.46
->boto3) (1.25.10)
Requirement already satisfied: six>=1.5 in /usr/lib/python3.9/site-packages (from python-dateutil<3.0.0,>=2.1->botocore<
1.39.0,>=1.38.46->boto3) (1.15.0)
[ec2-user@ip-172-31-84-183 TravelGo1]$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.84.183:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 594-419-608
223.185.81.172 - - [30/Jun/2025 12:27:17] "GET / HTTP/1.1" 200 -
223.185.81.172 - - [30/Jun/2025 12:27:17] "GET /static/images/flight-icon.jpeg HTTP/1.1" 304 -
223.185.81.172 - - [30/Jun/2025 12:27:17] "GET /static/images/train-icon.jpeg HTTP/1.1" 304 -
223.185.81.172 - - [30/Jun/2025 12:27:18] "GET /static/images/bus-icon.jpeg HTTP/1.1" 304 -
223.185.81.172 - - [30/Jun/2025 12:27:18] "GET /static/images/hotel.jpg HTTP/1.1" 304 -
223.185.81.172 - - [30/Jun/2025 12:27:53] "GET /login HTTP/1.1" 200 -
223.185.81.172 - - [30/Jun/2025 12:27:55] "GET /register HTTP/1.1" 200 -
223.185.81.172 - - [30/Jun/2025 12:28:13] "POST /register HTTP/1.1" 200 -
223.185.81.172 - - [30/Jun/2025 12:28:21] "GET /login HTTP/1.1" 200 -
223.185.81.172 - - [30/Jun/2025 12:28:29] "POST /login HTTP/1.1" 302 -
223.185.81.172 - - [30/Jun/2025 12:28:29] "GET /dashboard HTTP/1.1" 200 -
223.185.81.172 - - [30/Jun/2025 12:28:39] "POST /cancel_booking HTTP/1.1" 302 -
223.185.81.172 - - [30/Jun/2025 12:28:39] "GET /dashboard HTTP/1.1" 200 -
223.185.81.172 - - [30/Jun/2025 12:28:46] "GET /bus HTTP/1.1" 200 -
```

Milestone 8: Testing and Deployment

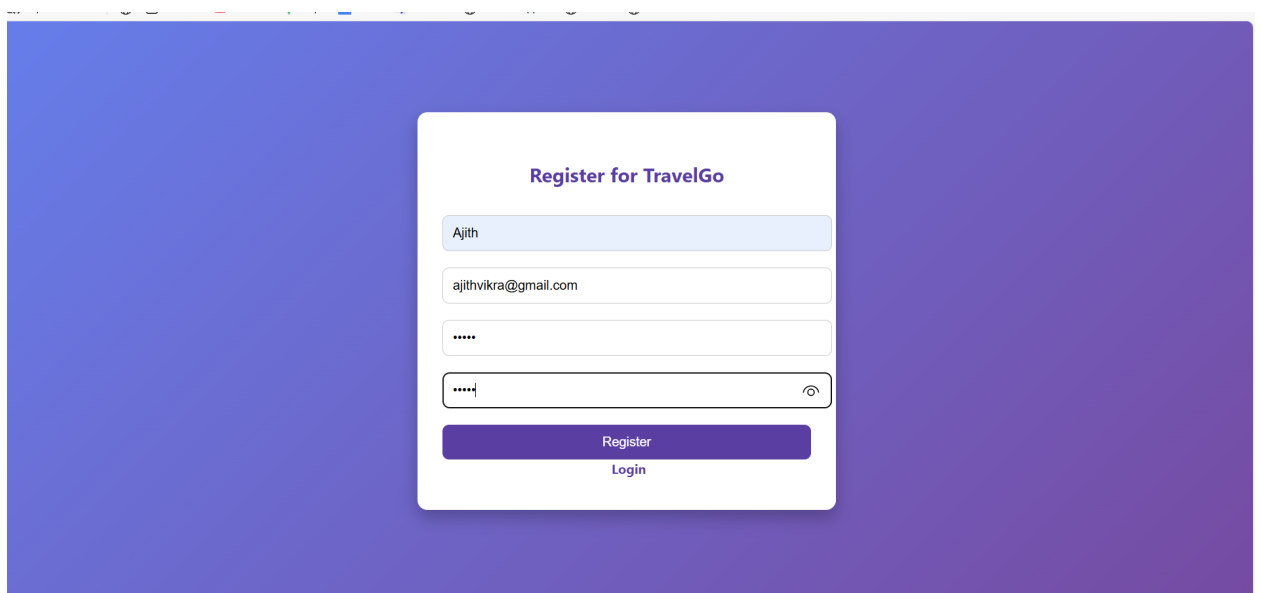
- **Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.**

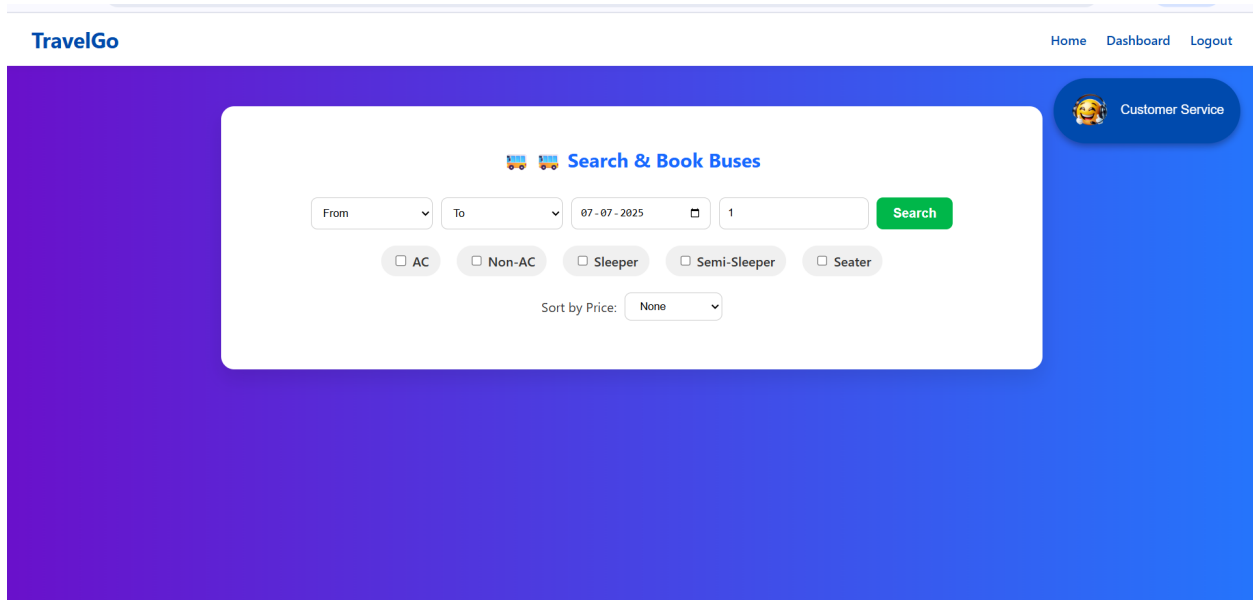
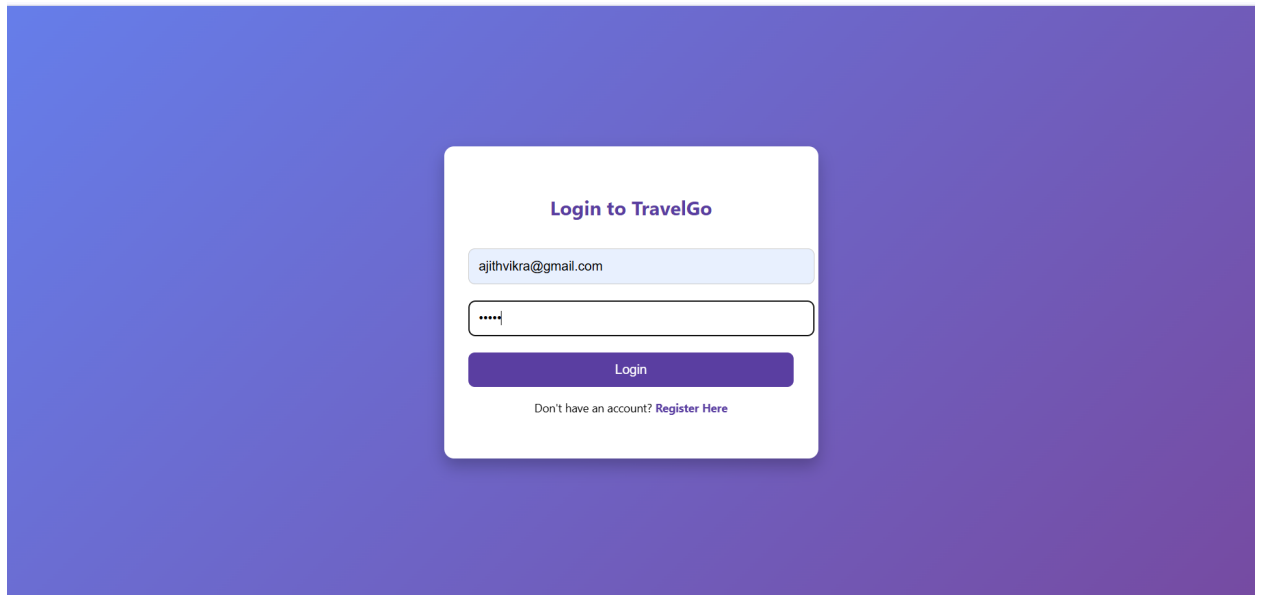


Welcome Page:



Register Page:





TravelGo

Hyderabad

Garuda Express
Non-AC Seater • 11:00 AM

Select Your Seats

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16
17	18	19	20
21	22	23	24
25	26	27	28
29	30	31	32
33	34	35	36
37	38	39	40
41	42	43	44
45	46	47	48
49	50	51	52

Confirm Cancel

Home Dashboard Logout

Search

Book

TravelGo

Confirm Bus Booking

Booking Details:

Bus Name: Garuda Express

Route: Hyderabad to Vijayawada

Date: 2025-07-07

Time: 11:00 AM

Type: Non-AC Seater

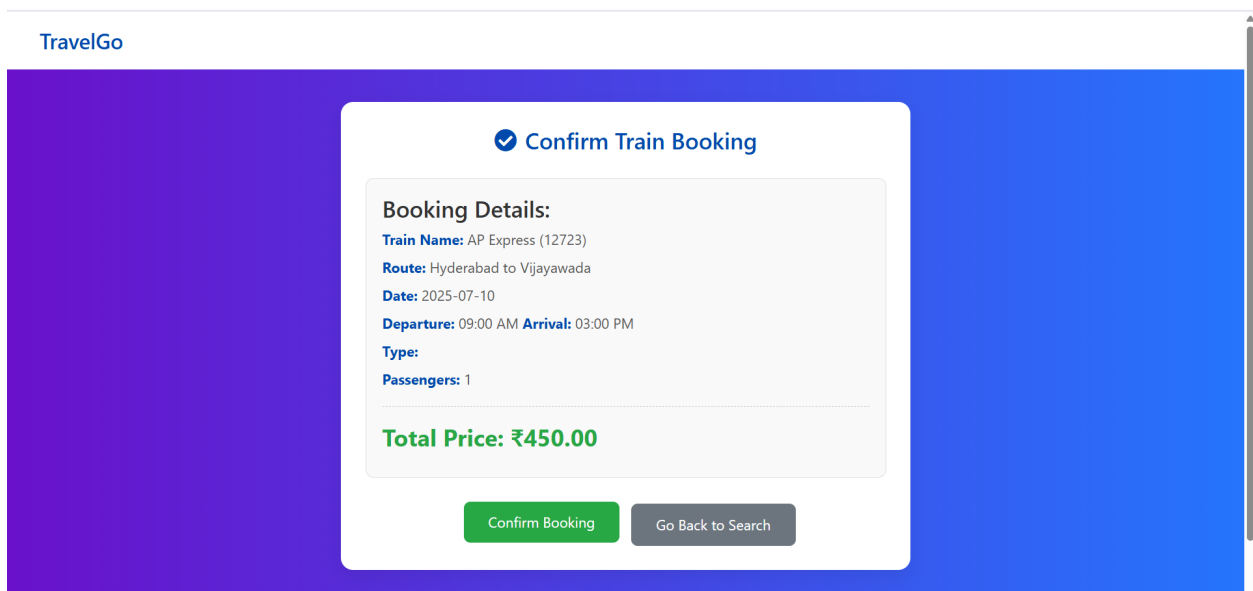
Passengers: 1

Selected Seats: 5

Total Price: ₹400.00

Confirm Booking

Go Back to Search



Flight Booking Page:

TravelGo

Home

Dashboard

Logout



Customer Service

✈️ Search & Book Flights

Hyderabad ▾ Mumbai ▾ 08-07-2025 📅 2

Economy ▾ **Search**

IndiGo 6E-234

Hyderabad → Mumbai

2025-07-08 08:00 - 09:30

Class: **Economy**

₹3500 x 2 = ₹7000

Book

TravelGo - Select Your Flight Seat

Select Your Seats

S1	S2	S3	S4
S5	S6	S7	S8
S9	S10	S11	S12
S13	S14	S15	S16
S17	S18	S19	S20
S21	S22	S23	S24
S25	S26	S27	S28
S29	S30	S31	S32
S33	S34	S35	S36
S37	S38	S39	S40
S41	S42	S43	S44
S45	S46	S47	S48
S49	S50	S51	S52

Confirm Booking

Cancel

Hotel Booking Page:

TravelGo

[Home](#) [Dashboard](#) [Logout](#)

 Customer Service

Find & Book Hotel Rooms

Hyderabad 08-07-2025 12-07-2025 1

2

Search

☐ 5-Star ☐ 4-Star ☐ 3-Star ☐ WiFi ☐ Pool ☐ Parking

Sort by:

Taj Falaknuma Palace

Hyderabad • 5-Star • ₹25000/night
Amenities: WiFi, Pool, Parking, Restaurant

Book

The Park

Hyderabad • 3-Star • ₹5500/night
Amenities: WiFi

Book

TravelGo

Confirm Your Hotel Booking

Hotel: Taj Falaknuma Palace

Location: Hyderabad

Check-in: 2025-07-08

Check-out: 2025-07-12

Rooms: 1

Guests: 2

Price/night: ₹25000.0

Total nights: 4

Confirm Booking

Go Back to Search

Dashboard:

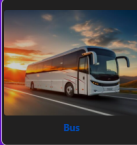
TravelGo

Home Dashboard Logout

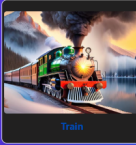
Welcome, test1@gmail.com!

Here you can view your upcoming and past travel bookings.

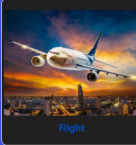
Hotel booking confirmed successfully!



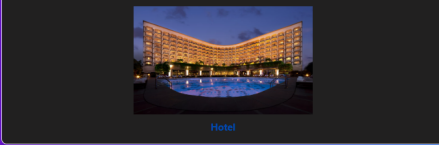
Bus



Train



Flight



Hotel

YOUR BOOKINGS

Customer Service

YOUR BOOKINGS

Booking: Taj Falaknuma Palace

Location: Hyderabad
Check-in: 2025-07-08
Check-out: 2025-07-12
Rooms: 1
Guests: 2
Booked On: 2025-07-07

₹100,000.00

Cancel Booking

Bookings:

Type: flight
Route: Hyderabad → Mumbai
Date of Travel: 2025-07-08
Seats: 514
Departure: 08:00
Arrival: 09:30
Passengers: 1
Class: Economy
Booked On: 2025-07-07

₹3,500.00

Cancel Booking

Booking: Kaveri Travels

Type: bus
Route: Hyderabad → Guntur
Date of Travel: 2025-07-07
Seats: 23
Time: 09:30 AM
Passengers: 1
Booked On: 2025-07-07

₹550.00

Cancel Booking

Booking: Orange Travels

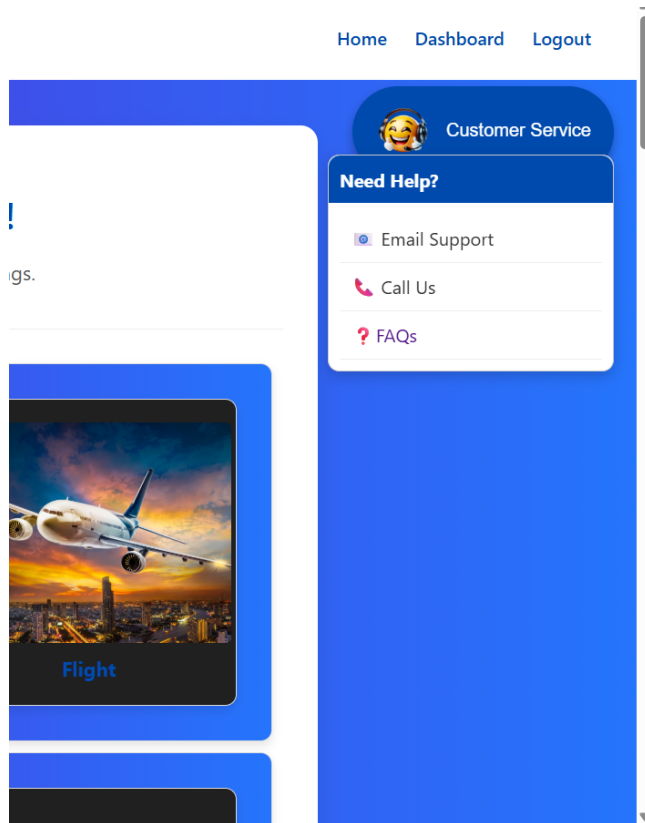
Type: bus
Route: Hyderabad → Vijayawada
Date of Travel: 2025-07-08
Time: 05:00 AM
Passengers: 1
Booked On: 2025-07-07

₹800.00

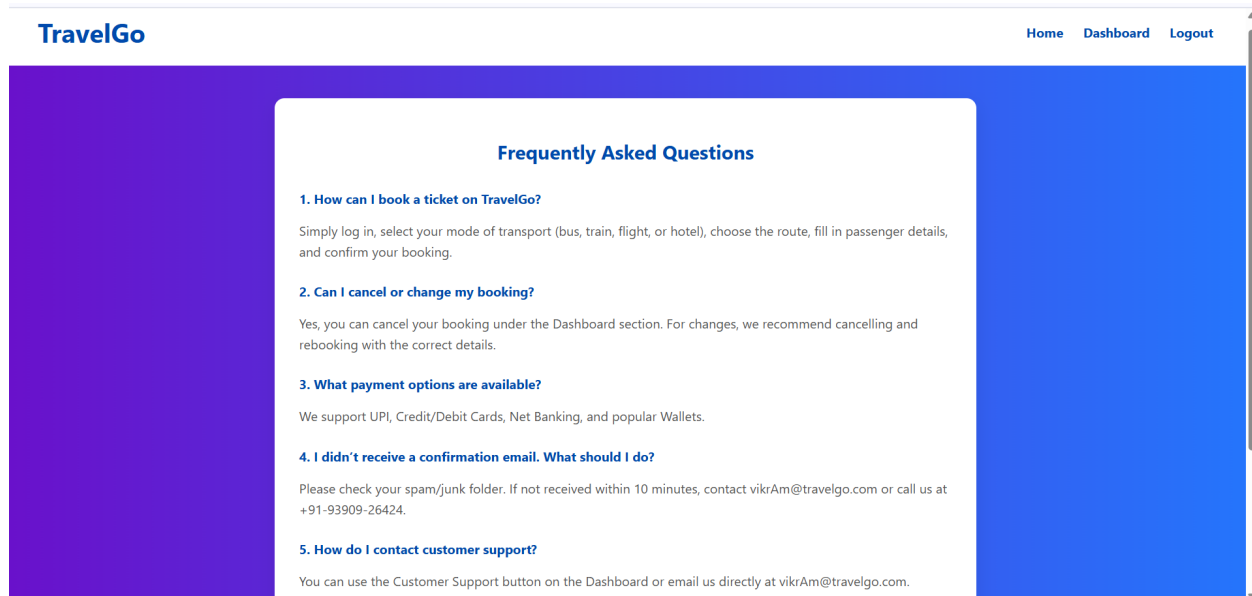
Cancel Booking

Customer Service

Customer Service:



Frequently Asked Questions:





Conclusion:

The **TravelGo** Website has been successfully developed and deployed using a scalable and cloud-native architecture. Leveraging AWS services such as EC2 for hosting, DynamoDB for real-time data management, and SNS for instant booking and cancellation notifications, the platform provides a seamless travel booking experience for users. TravelGo enables registered users to search and book buses, trains, flights, and hotels in a centralized, intuitive interface, eliminating the complexities of navigating multiple travel services.

The cloud infrastructure ensures high availability and smooth performance even during peak usage, while the Flask backend ensures efficient handling of user authentication, dynamic booking flows, and data transactions. Real-time notification integration via AWS SNS allows users to receive booking confirmations and cancellations immediately via email, improving communication and user engagement.

In summary, the **TravelGo** Website offers a modern, reliable, and user-friendly solution for managing travel and accommodation needs. It highlights the potential of cloud-based platforms in building unified travel systems, simplifying operations, and enhancing the overall user experience.

COMMENTS

Document Name : SI-27245-1751640420

C1 "I" in Page 1

Guest #3 on Monday, Jul 07, 2025, 07:08 AM

Pusuluri Ajith KHIT CSE 228x1a0583@khitguntur.ac.in